claude-windows / c47f0851-d75f-45b6-9126-e48544381426

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\c47f0851-d75f-45b6-9126-e48544381426\subagents\workflows\wf_5af9a132-4ce\agent-aef5af4636f4180e9.jsonl`
- SHA-256: `5d47e411065d4307c1d5989d9830a052ff4b03f15ba73a8ece0ea04b8b1d1a9c`
- Source modified: `2026-06-20T09:00:59+00:00`
- Imported at: `2026-07-05T16:48:23+00:00`
- project: `wf_5af9a132-4ce`
- session_id: `c47f0851-d75f-45b6-9126-e48544381426`

Transcript

1. user (2026-06-20T08:57:11.748Z)

```
Repo: badminton-highlight-indexer (Python). Branch feat/native-wasb-runner off origin/master.
The change builds the M3.5 Linux-native WASB detector runner + wires real extraction into the
worker.
Files changed/added (read them in the WORKING TREE ? changes are uncommitted):
- backend/pipeline/detectors/native_wasb_runner.py  (NEW ? the native runner)
- backend/pipeline/detectors/wasb_infer.py           (added --device, threaded device into
_build_cfg/run/run_video_streaming/main; device-keyed the resumable manifest)
- backend/pipeline/segmenters/trajectory_hybrid.py   (_resolve_runner: new 'native' branch +
_build_native_runner base hook; generalized 'WSL env' log lines)
- backend/pipeline/segmenters/wasb_hybrid.py         (_build_native_runner override)
- backend/pipeline/segmenters/fusion_hybrid.py       (_build_native_runner override; WASB
substrate only)
- backend/config/models.py                            (WasbIndexConfig.device + .python_bin;
detector_impl doc mentions 'native')
- backend/worker/__main__.py                          (cmd_train: --trajectory-source
{synth,detector} + --segmenter; _resolve_train_detector; detector path pulls videos + runs
runner)
- tests/test_native_wasb_runner.py, tests/test_worker.py (new tests)

DESIGN INTENT / CONTRACTS (judge against these):
- The EXISTING WSL runner is backend/pipeline/detectors/wasb_runner.py (WasbRunner,
WslCommandMixin). The native runner must be a sibling that runs the SAME wasb_infer.py natively
(no wsl, no 'conda activate' ? it invokes the wasb conda env python BY PATH, no /mnt path
translation) and emit a BYTE-IDENTICAL Frame,Visibility,X,Y CSV with the SAME filename
'{stem}__{vid12}_wasb.csv'.
- ACCEPTANCE GATE = byte-parity: same clip via WSL (Windows) vs native (Linux, device=cuda) ->
identical CSV + identical candidate windows. device default 'cuda' must keep wasb_infer's Hydra
overrides identical to before (parity).
- Selected via the detector_impl seam: indexing.detector_impl / RALLY_DETECTOR_IMPL = 'native'.
- Constraint: 'backend' must NOT import 'training' (worker shells out). The default trajectory
source stays 'synth' so the CPU e2e is unchanged.
- The code RUNS on Linux (GPU box) but the TESTS run on the dev's Windows machine; cross-
platform path handling matters.
- weights/repo/python/scratch come from env first
(WASB_WEIGHTS/WASB_REPO_DIR/WASB_PYTHON/WASB_SCRATCH) then indexing.wasb.* then default.

YOU ARE THE BYTE-PARITY SKEPTIC. Read native_wasb_runner.py and wasb_infer.py (changed) and
compare against wasb_runner.py line-by-line. Find ANY way the native path could produce a
DIFFERENT trajectory CSV than the WSL path on device=cuda: differences in the argv passed to
wasb_infer.py, the out-CSV name/path, the cwd, frame extraction vs streaming defaults, the
copied infer script, the Hydra override syntax change in _build_cfg (runner.gpus=[0] for cuda vs
[]), the device-keyed manifest back-compat (does an OLD cuda cache with no 'device' field still
resume on a cuda run? does a cpu run wrongly reuse a cuda cache?). Also: does adding
--device/device params break the existing pure-Python cache tests? Report concrete divergences
```

only.

2. assistant (2026-06-20T08:57:14.333Z)

I'll analyze the byte-parity contract between the native and WSL WASB runners. Let me start by reading the relevant files in the working tree.

3. user (2026-06-20T08:57:15.766Z)

```
1       """Linux-native WASB shuttle-trajectory runner (no WSL).
2
3       The de-risk goal the ``DetectorRunner`` ABC was built for (see ``base.py`` ?
4       "Optional Linux-native path"; ``docs/DECOUPLED_COMPUTE/04-BUILD-AND-TEST-PLAN.md``
M3.5).
5       It is the sibling of :class:`~backend.pipeline.detectors.wasb_runner.WasbRunner` for a
6       Linux GPU box (GCP / a SaaS worker): it runs the **same** verified ``wasb_infer.py``
core
7       that the WSL adapter runs, so the trajectory CSV is byte-identical ? but WITHOUT the WSL
8       transport:
9
10        * NO ``WslCommandMixin`` (no ``wsl -d <distro> bash -c``), NO ``conda activate`` (the
11          ``wasb`` env's python is invoked **by path**), NO ``/mnt/c`` path translation
12          (``to_wsl_mnt_path``). Paths are native; the video is already local on the box.
13        * ``device`` is ``cuda`` (default ? the WSL parity path), ``mps`` or ``cpu``; threaded
14          into ``wasb_infer.py --device`` (and so into the Hydra ``runner.device`` override).
15        * weights / repo / python / scratch come from the **environment** first
16          (``WASB_WEIGHTS`` / ``WASB_REPO_DIR`` / ``WASB_PYTHON`` / ``WASB_SCRATCH``), then
17          ``indexing.wasb.*`` config, then a default ? so a box needs no config edits.
18
19      It emits the identical ``Frame,Visibility,X,Y`` CSV (same ``{stem}__{vid12}_wasb.csv``
20      name as the WSL runner) and reuses the model-agnostic ``parse_trajectory_csv`` /
21      ``trajectory_to_action_windows`` from ``tracknet_runner``, so the trajectory hybrids and
22      the windowing brain are unchanged. Selected via the existing ``detector_impl`` seam
23      (``trajectory_hybrid._resolve_runner`` ? ``_build_native_runner``);
``detector_impl="native"``.
24
25      **Acceptance gate (owner / GPU box):** byte-parity ? the SAME clip through the WSL
runner
26      (Windows) and this runner (Linux, ``device=cuda``) yields an identical trajectory CSV
and
27      identical candidate windows. That parity is a hardware check (a real GPU + the ``wasb``
28      env), so it is verified by the owner on the box ? not in this offline, subprocess-mocked
29      test suite.
30      """
31      import os
32      import shutil
33      import logging
34      import subprocess
35      from dataclasses import dataclass
36      from typing import Any, Dict, List, Optional, Tuple
37
38      from backend.pipeline.detectors.base import DetectorRunner
39      # Reuse the model-agnostic helpers ? trajectory shape + windowing are identical to
WSL/TrackNet.
40      from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
41
42      logger = logging.getLogger(__name__)
43
44      # Path to the inference core we copy into the WASB repo's src/ (so it can import WASB's
45      # detectors/trackers/dataloaders + find configs/), exactly as the WSL adapter does.
46      _INFER_PY = os.path.join(os.path.dirname(os.path.abspath(__file__)), "wasb_infer.py")
47
48
49      @dataclass
50      class NativeWasbConfig:
```

```
51          """Resolved settings for a Linux-native WASB run.
52
53          Built via :meth:`from_indexing_cfg` with **environment overrides** taking precedence
54          over ``indexing.wasb.*`` config (the box sets env; no config edits needed).
``device``
55          defaults to ``cuda`` (the WSL byte-parity path).
56          """
57          repo_dir: str = "~/models/WASB-SBDT"   # native clone of nttcom/WASB-SBDT
58          python_bin: str = "python"             # the `wasb` conda env python (invoked by
path, no activate)
59          weights_path: str = ""                 # REQUIRED (env WASB_WEIGHTS or
indexing.wasb.weights_path)
60          sport: str = "badminton"
61          device: str = "cuda"                   # cuda | mps | cpu
62          scratch_dir: str = ""                  # frame-extraction scratch (env); "" = next
to the output dir
63          timeout_sec: int = 1800               # 30 min; a hung GPU call is killed (0 = no
timeout)
64          keep_frames: bool = False              # retain the decoded frame cache after
success (debug only)
65          stream_video: bool = False             # fast path: decode in-process, no PNG
extraction (bit-identical)
66
67          @classmethod
68          def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "NativeWasbConfig":
69              w = (idx_cfg or {}).get("wasb", {}) or {}
70              env = os.environ.get
71              return cls(
72                  repo_dir=env("WASB_REPO_DIR") or w.get("repo_dir", "~/models/WASB-SBDT"),
73                  python_bin=env("WASB_PYTHON") or w.get("python_bin", "python"),
74                  weights_path=env("WASB_WEIGHTS") or w.get("weights_path", "") or "",
75                  sport=w.get("sport", "badminton"),
76                  device=env("WASB_DEVICE") or w.get("device", "cuda"),
77                  scratch_dir=env("WASB_SCRATCH") or env("RALLY_SCRATCH") or env("TMPDIR") or
"",
78                  timeout_sec=int(w.get("timeout_sec", 1800)),
79                  keep_frames=bool(w.get("keep_frames", False)),
80                  stream_video=bool(w.get("stream_video", False)),
81              )
82
83
84      class NativeWasbRunner(DetectorRunner):
85          """WASB runner that executes natively on a Linux GPU box ? no WSL. See module
docstring."""
86
87          def __init__(self, cfg: NativeWasbConfig):
88              self.cfg = cfg
89
90          # --- low-level native exec (the single place tests patch)
--------------------------
91          def _run(self, argv: List[str], cwd: Optional[str] = None) ->
subprocess.CompletedProcess:
92              logger.debug("native exec: %s (cwd=%s)", " ".join(argv), cwd)
93              return subprocess.run(argv, cwd=cwd, capture_output=True, text=True,
94                                    timeout=(self.cfg.timeout_sec or None))
95
96          def _repo_src(self) -> str:
97              return os.path.join(os.path.expanduser(self.cfg.repo_dir), "src")
98
99          def _copy_infer_script(self) -> bool:
100             """Copy wasb_infer.py into the WASB repo src/ so it imports WASB modules + finds
configs/."""
101             repo_src = self._repo_src()
102             try:
103                 os.makedirs(repo_src, exist_ok=True)
```

```
104                 shutil.copy(_INFER_PY, os.path.join(repo_src, "wasb_infer.py"))
105                 return True
106             except OSError as e:
107                 logger.error("Failed to copy wasb_infer.py into %s: %s", repo_src, e)
108                 return False
109 
110         def _rm_rf(self, path: Optional[str]) -> None:
111             """Best-effort recursive delete of a native path (post-success cleanup; never
raises)."""
112             if path:
113                 shutil.rmtree(path, ignore_errors=True)
114 
115         def healthcheck(self) -> Tuple[bool, str]:
116             """Verify weights + repo present and the `wasb` python imports torch (and sees a
GPU on cuda)."""
117             if not self.cfg.weights_path:
118                 return False, ("WASB weights path is empty ? set WASB_WEIGHTS (or
indexing.wasb.weights_path).")
119             weights = os.path.expanduser(self.cfg.weights_path)
120             if not os.path.exists(weights):
121                 return False, f"WASB weights not found at {weights}"
122             repo_src = self._repo_src()
123             if not os.path.isdir(repo_src):
124                 return False, (f"WASB-SBDT repo src/ not found at {repo_src} ? clone
nttcom/WASB-SBDT "
125                                f"(set WASB_REPO_DIR / indexing.wasb.repo_dir).")
126             code = "import torch; print('torch', torch.__version__, 'cuda',
torch.cuda.is_available())"
127             try:
128                 res = self._run([self.cfg.python_bin, "-c", code])
129             except FileNotFoundError:
130                 return False, f"python binary not found: {self.cfg.python_bin!r} (set
WASB_PYTHON)."
131             except subprocess.TimeoutExpired:
132                 return False, "native torch healthcheck timed out."
133             out = (res.stdout or "").strip()
134             if res.returncode != 0:
135                 return False, f"`{self.cfg.python_bin}` torch import failed: {(res.stderr or
out).strip()}"
136             if self.cfg.device == "cuda" and "cuda True" not in out:
137                 return False, f"device=cuda but torch.cuda.is_available() is False ({out})"
138             return True, out
139 
140         def run_predict(self, video_win_path: str, output_win_dir: str,
141                         video_id: Optional[str] = None) -> Optional[str]:
142             """Run WASB natively on a video. Returns the path to the trajectory CSV, or
None.
143 
144             Output artifacts are namespaced by ``video_id`` (the file MD5) exactly as the
WSL
145             runner does, and the CSV is named ``{stem}__{vid12}_wasb.csv`` ? byte-for-byte
the
146             same downstream contract, so the parity comparison is apples-to-apples.
147             """
148             output_dir = os.path.abspath(output_win_dir)
149             os.makedirs(output_dir, exist_ok=True)
150             # Normalize for a cross-platform basename (tests pass Windows paths on Linux).
151             norm = str(video_win_path).replace("\\", "/")
152             stem, _ext = os.path.splitext(os.path.basename(norm))
153             # Content-derived key: same filename + different bytes -> different key (no
cache reuse).
154             key = f"{stem}__{video_id[:12]}" if video_id else (stem or "wasb")
155             out_csv = os.path.join(output_dir, f"{key}_wasb.csv")
156 
157             if not self._copy_infer_script():
```

```
158                 logger.error("Could not stage wasb_infer.py into the WASB repo src/.")
159                 return None
160
161             weights = os.path.expanduser(self.cfg.weights_path)
162             argv = [self.cfg.python_bin, "wasb_infer.py",
163                     "--video", os.path.abspath(str(video_win_path))]
164             frames_dir: Optional[str] = None
165             if self.cfg.stream_video:
166                 # Fast path: decode in-process, no PNG extraction. Output is bit-identical
to disk.
167                 argv.append("--stream-video")
168             else:
169                 scratch = self.cfg.scratch_dir or output_dir
170                 frames_dir = os.path.join(scratch, f"{key}_frames")
171                 argv += ["--frames_out_dir", frames_dir]
172             argv += ["--weights", weights,
173                      "--sport", self.cfg.sport,
174                      "--device", self.cfg.device,
175                      "--out", out_csv]
176
177             logger.info("Running native WASB inference (device=%s) on %s ...",
178                         self.cfg.device, os.path.basename(norm))
179             try:
180                 res = self._run(argv, cwd=self._repo_src())
181             except FileNotFoundError:
182                 logger.error("python binary not found: %r (set WASB_PYTHON).",
self.cfg.python_bin)
183                 return None
184             except subprocess.TimeoutExpired:
185                 logger.error("WASB native inference timed out after %ss.",
self.cfg.timeout_sec)
186                 return None
187             if res.returncode != 0:
188                 logger.error("WASB native inference failed:\n%s", (res.stderr or res.stdout
or "")[-2000:])
189                 return None
190             if not os.path.exists(out_csv):
191                 logger.error("WASB native finished but expected CSV not found at %s",
out_csv)
192                 return None
193             logger.info("WASB trajectory CSV: %s", out_csv)
194
195             # Success ? the CSV is the durable output; the decoded frame cache is a pure
intermediate.
196             # On any earlier failure we return above WITHOUT cleaning, so the cache survives
a resume.
197             # (Streaming never materializes frames, so there is nothing to drop on that
path.)
198             if not self.cfg.keep_frames and frames_dir:
199                 logger.info("Cache hygiene: removing frame cache for %s "
200                             "(set indexing.wasb.keep_frames=true to retain).", key)
201                 self._rm_rf(frames_dir)
202             return out_csv
203
204         # Trajectory parsing + windowing are inherited from tracknet_runner (model-
agnostic).
205         parse_trajectory_csv = staticmethod(TrackNetRunner.parse_trajectory_csv)
206         trajectory_to_action_windows =
staticmethod(TrackNetRunner.trajectory_to_action_windows)
207
```

4. user (2026-06-20T08:57:16.230Z)

```
1       """WASB shuttle-trajectory runner (WSL2 subprocess adapter).
2
```

```
3       Windows-side counterpart to backend/detectors/wasb_infer.py. It:
4         1. stages the video into the WSL filesystem (fast local reads, not /mnt),
5         2. copies wasb_infer.py into the WASB repo's src/ dir,
6         3. runs it in the WSL `wasb` conda env -> a Frame,Visibility,X,Y trajectory CSV,
7         4. reuses the (model-agnostic) trajectory->action-window logic from tracknet_runner.
8
9       So the only WASB-specific bits live here; the windowing + CSV parsing are shared
10      with the TrackNet path. Verified inference core: wasb_infer.py (see its docstring).
11      """
12      import os
13      import shlex
14      import logging
15      import subprocess
16      from dataclasses import dataclass
17      from typing import Dict, Any, Optional, Tuple
18
19      # Reuse the model-agnostic helpers ? trajectory shape + windowing are identical.
20      # parse_trajectory_csv / trajectory_to_action_windows are @staticmethods on
TrackNetRunner.
21      from backend.pipeline.detectors.tracknet_runner import to_wsl_mnt_path, TrackNetRunner
22      from backend.pipeline.detectors.base import DetectorRunner, WslCommandMixin,
wsl_tilde_quote
23
24      logger = logging.getLogger(__name__)
25
26      # Windows path to the inference wrapper we stage into WSL.
27      _INFER_WIN = os.path.join(os.path.dirname(os.path.abspath(__file__)), "wasb_infer.py")
28
29
30      @dataclass
31      class WasbConfig:
32          """Resolved settings for a WASB WSL run (built from config.json ->
indexing.wasb)."""
33          distro: str = "Ubuntu"
34          repo_dir: str = "~/models/WASB-SBDT"
35          conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
36          conda_env: str = "wasb"
37          weights_path: str = "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
38          sport: str = "badminton"
39          wsl_stage_dir: str = "~/clips"
40          timeout_sec: int = 1800  # 30 min; a hung GPU/conda call is killed (0 = no timeout)
41          keep_frames: bool = False  # retain staged video + frame cache after success (debug
only)
42          min_free_gb: float = 0.0   # warn if WSL free space below this before a run (0 =
off)
43          # FAST PATH (default OFF until verified): decode the staged video on the fly inside
44          # WSL and feed frames straight to the detector, skipping the slow per-frame PNG
45          # extraction that dominates a long clip. Output is bit-identical to the PNG path
46          # (PNG is lossless), but this stays opt-in until the owner's side-by-side confirms
it.
47          stream_video: bool = False
48
49          @classmethod
50          def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "WasbConfig":
51              w = (idx_cfg or {}).get("wasb", {}) or {}
52              return cls(
53                  distro=w.get("wsl_distro", "Ubuntu"),
54                  repo_dir=w.get("repo_dir", "~/models/WASB-SBDT"),
55                  conda_sh=w.get("conda_sh", "~/miniconda3/etc/profile.d/conda.sh"),
56                  conda_env=w.get("conda_env", "wasb"),
57                  weights_path=w.get("weights_path",
58                                     "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"),
59                  sport=w.get("sport", "badminton"),
```

```
                wsl_stage_dir=w.get("wsl_stage_dir", "~/clips"),
                timeout_sec=int(w.get("timeout_sec", 1800)),
                keep_frames=bool(w.get("keep_frames", False)),
                min_free_gb=float(w.get("min_free_gb", 0.0)),
                stream_video=bool(w.get("stream_video", False)),
            )


    class WasbRunner(WslCommandMixin, DetectorRunner):
        def __init__(self, cfg: WasbConfig):
            self.cfg = cfg

        def healthcheck(self) -> Tuple[bool, str]:
            """Verify the WSL `wasb` env imports torch and sees a GPU. Returns (ok, msg)."""
            cmd = (f"conda activate {self.cfg.conda_env} && "
                   f"python -c \"import torch; print('torch', torch.__version__, "
                   f"'cuda', torch.cuda.is_available())\"")
            try:
                res = self._wsl_bash(cmd)
            except FileNotFoundError:
                return False, "`wsl` not found ? Windows + WSL2 required."
            except subprocess.TimeoutExpired:
                return False, "WSL healthcheck timed out."
            out = (res.stdout or "").strip()
            if res.returncode != 0:
                return False, f"WSL `wasb` env check failed: {(res.stderr or out).strip()}"
            return True, out

        def _stage_video(self, video_win_path: str, dest_name: str) -> Optional[str]:
            src_mnt = to_wsl_mnt_path(video_win_path)
            reason = self.wsl_source_unreadable_reason(src_mnt)
            if reason:
                logger.error("Cannot stage video into WSL: %s", reason)
                return None
            dest = f"{self.cfg.wsl_stage_dir}/{dest_name}"
            cmd = f"mkdir -p {self.cfg.wsl_stage_dir} && cp {shlex.quote(src_mnt)}
{wsl_tilde_quote(dest)} && echo OK"
            try:
                res = self._wsl_bash(cmd)
            except subprocess.TimeoutExpired:
                logger.error("Staging video into WSL timed out.")
                return None
            if res.returncode != 0 or "OK" not in (res.stdout or ""):
                logger.error("Failed to stage video into WSL: %s", (res.stderr or
res.stdout).strip())
                return None
            return dest

        def _stage_infer_script(self) -> bool:
            """Copy wasb_infer.py into the WASB repo src/ so it can import WASB modules."""
            src_mnt = to_wsl_mnt_path(_INFER_WIN)
            cmd = f"cp {shlex.quote(src_mnt)} {self.cfg.repo_dir}/src/wasb_infer.py && echo
OK"
            res = self._wsl_bash(cmd)
            return res.returncode == 0 and "OK" in (res.stdout or "")

        def run_predict(self, video_win_path: str, output_win_dir: str,
                        video_id: Optional[str] = None) -> Optional[str]:
            """Run WASB on a video. Returns the Windows path to the trajectory CSV, or None.

            All WSL/cache/output artifacts are namespaced by ``video_id`` (the file MD5) so
            two videos that share a filename (e.g. two ``match.mp4`` from different folders)
            can never reuse each other's staged frames, resumable cache, or CSV.
            """
            # Resolve relative dirs (the hybrids pass "output/wasb") BEFORE the /mnt
```

```
            # translation: to_wsl_mnt_path passes relative paths through unchanged, so
            # WSL resolved them against the inference cwd (~/models/WASB-SBDT/src) and
            # the CSV landed inside WSL while Windows polled the repo-relative path.
            output_win_dir = os.path.abspath(output_win_dir)
            os.makedirs(output_win_dir, exist_ok=True)
            # Normalize for cross-platform basename (tests use Windows paths on Linux).
            video_win_path = str(video_win_path).replace("\\", "/")
            base = os.path.basename(video_win_path)
            stem, ext = os.path.splitext(base)
            # Unique, content-derived key: same filename + different bytes -> different key.
            key = f"{stem}__{video_id[:12]}" if video_id else stem
            wsl_out_dir = to_wsl_mnt_path(output_win_dir)
            expected_csv_win = os.path.join(output_win_dir, f"{key}_wasb.csv")

            if not self._stage_infer_script():
                logger.error("Could not stage wasb_infer.py into WSL.")
                return None
            staged_video = self._stage_video(video_win_path, f"{key}{ext}")
            if staged_video is None:
                return None
            frames_dir = f"{self.cfg.wsl_stage_dir}/{key}_frames"

            # Disk guard: a 4K clip can extract ~100 GB of PNG frames. Warn early if the
            # WSL stage filesystem is already low so the user can clean up before it fills.
            # (Streaming never materializes the PNGs, so the guard only matters off the fast
path.)
            if self.cfg.min_free_gb > 0 and not self.cfg.stream_video:
                free = self._wsl_free_gb(self.cfg.wsl_stage_dir)
                if free is not None and free < self.cfg.min_free_gb:
                    logger.warning(
                        "Low WSL disk: %.1f GB free at %s (< min_free_gb=%.1f). Frame
extraction "
                        "may fill the disk ? consider running tools/cleanup_caches.py.",
                        free, self.cfg.wsl_stage_dir, self.cfg.min_free_gb)

            # Fast path: stream-decode the video in-process (no PNG extraction). Output is
            # bit-identical to the PNG round-trip but avoids writing ~46k PNGs for a 12-min
clip.
            if self.cfg.stream_video:
                frame_args = "--stream-video "
            else:
                frame_args = f"--frames_out_dir {wsl_tilde_quote(frames_dir)} "
            cmd = (
                f"conda activate {self.cfg.conda_env} && cd {self.cfg.repo_dir}/src && "
                f"python wasb_infer.py "
                f"--video {wsl_tilde_quote(staged_video)} "
                f"{frame_args}"
                f"--weights {wsl_tilde_quote(self.cfg.weights_path)} "
                f"--sport {shlex.quote(self.cfg.sport)} "
                f"--out {wsl_tilde_quote(wsl_out_dir + '/' + key + '_wasb.csv')}"
            )
            logger.info("Running WASB inference on %s ...", base)
            try:
                res = self._wsl_bash(cmd)
            except subprocess.TimeoutExpired:
                logger.error("WASB inference timed out after %ss.", self.cfg.timeout_sec)
                return None
            if res.returncode != 0:
                logger.error("WASB inference failed:\n%s", (res.stderr or
res.stdout)[-2000:])
                return None
            if not os.path.exists(expected_csv_win):
                logger.error("WASB finished but expected CSV not found at %s",
expected_csv_win)
                return None
```

```
182             logger.info("WASB trajectory CSV: %s", expected_csv_win)
183
184             # Success ? the trajectory CSV is the durable output; the staged video copy and
185             # the (huge) decoded frame cache are now pure intermediates. Delete them unless
186             # explicitly retained. On any earlier failure we return above WITHOUT cleaning,
187             # so the frame cache + detector cache remain for a resumable re-run.
188             # (Streaming never writes a frame cache, so there's only the staged video to
drop.)
189             if not self.cfg.keep_frames:
190                 stale = [staged_video] if self.cfg.stream_video else [frames_dir,
staged_video]
191                 logger.info("Cache hygiene: removing staged video%s for %s "
192                             "(set indexing.wasb.keep_frames=true to retain).",
193                             "" if self.cfg.stream_video else " + frame cache", key)
194                 self._wsl_rm_rf(stale)
195             return expected_csv_win
196
197         # Trajectory parsing + windowing are inherited from tracknet_runner (model-
agnostic).
198         parse_trajectory_csv = staticmethod(TrackNetRunner.parse_trajectory_csv)
199         trajectory_to_action_windows =
staticmethod(TrackNetRunner.trajectory_to_action_windows)
200
```

5. user (2026-06-20T08:57:17.176Z)

```
1       """WASB shuttle-trajectory inference on an arbitrary frame sequence (runs in WSL).
2
3       WASB-SBDT only ships *dataset* evaluation; this is a thin wrapper that reuses its
4       own building blocks (ImageDataset transform + detector + online tracker) to run on
5       any folder of extracted frames and emit a trajectory CSV in the exact shape the
6       indexer's `tracknet_runner.parse_trajectory_csv` already consumes:
7
8           Frame,Visibility,X,Y
9
10      It must run INSIDE the WSL `wasb` conda env, from the WASB-SBDT `src/` dir (it
11      imports WASB's `detectors`/`trackers`/`dataloaders`). The Windows-side adapter
12      (`wasb_runner.py`) stages this file + the frames into WSL and invokes it.
13
14      Resumable across reboots
15      ------------------------
16      The GPU detector pass (one forward per sliding window ? ~21k for a 6-min clip) is
17      the slow, expensive part; the CPU tracker pass that turns detections into a
18      trajectory is cheap and *stateful* (it must see every frame in order, so it can't
19      resume mid-stream). We exploit that split:
20
21        * The detector pass writes its per-window raw outputs incrementally to
22          ``<cache>/det_raw.jsonl`` (flushed+fsync'd every ``--flush-every`` windows) and
23          records progress in ``<cache>/manifest.json`` (atomic write). A reboot loses at
24          most ``--flush-every`` windows of GPU work instead of the whole pass.
25        * On restart we replay the cached windows, skip the DataLoader ahead to the first
26          un-cached window, and continue. Once every window is cached, the detector pass
27          is skipped entirely and we just re-run the (cheap, deterministic) tracker over
28          the full ordered cache -> trajectory CSV. So a lost trajectory CSV costs seconds,
29          not the GPU hour.
30
31      The cache is keyed by the output path (``<out>_wasbcache/`` by default), lives next
32      to the output (NOT in %TEMP%), and is invalidated automatically if the frames dir,
33      weights, sport, window size, or frame count change.
34
35      Usage (in WSL, from ~/models/WASB-SBDT/src):
36          python wasb_infer.py --frames_dir /path/frames --weights
../pretrained_weights/wasb_badminton_best.pth.tar \
37              --out /path/traj.csv [--sport badminton] [--limit N] [--cache-dir DIR] [--flush-
every 1000] [--fresh]
```

```
    """
    import os
    import os.path as osp
    import csv
    import glob
    import json
    import argparse
    import logging
    from collections import defaultdict
    from contextlib import contextmanager

    logger = logging.getLogger(__name__)

    CACHE_VERSION = 1
    _DET_FILE = "det_raw.jsonl"
    _MANIFEST_FILE = "manifest.json"


    def _build_cfg(weights: str, sport: str, device: str = "cuda"):
        """Compose the WASB eval config via Hydra, overriding model/weights/device.

        ``device`` is ``cuda`` (default ? the historical WSL behaviour, byte-parity
preserved),
        ``mps`` (Apple) or ``cpu``. Only the single-GPU CUDA path requests
``runner.gpus=[0]``;
        cpu/mps run with no GPU list so the detector places tensors on ``device``.
        """
        from hydra import compose, initialize
        gpus = "[0]" if device == "cuda" else "[]"   # single GPU on cuda; none for cpu/mps
        with initialize(config_path="configs", version_base=None):
            cfg = compose(config_name="eval", overrides=[
                f"dataset={sport}",
                "model=wasb",
                f"detector.model_path={weights}",
                f"runner.device={device}",
                f"runner.gpus={gpus}",  # default config assumes multiple GPUs; pin to this
box
            ])
        return cfg


    def _frame_id(path: str) -> int:
        """Best-effort integer frame id from a frame filename (e.g. frame_000180.png ->
180)."""
        stem = osp.splitext(osp.basename(path))[0]
        digits = "".join(ch for ch in stem if ch.isdigit())
        return int(digits) if digits else -1


    # ---------------------------------------------------------------------- #
    # Resumable detector cache (pure-Python, no torch/numpy ? unit-testable)
    # ---------------------------------------------------------------------- #
    def _cache_paths(cache_dir: str):
        return osp.join(cache_dir, _DET_FILE), osp.join(cache_dir, _MANIFEST_FILE)


    def default_cache_dir(out_csv: str) -> str:
        """Stable cache dir next to the trajectory output (survives reboots; not %TEMP%)."""
        d = osp.dirname(osp.abspath(out_csv))
        stem = osp.splitext(osp.basename(out_csv))[0]
        return osp.join(d, stem + "_wasbcache")


    def _atomic_write_text(path: str, text: str) -> None:
        """Write then rename so a crash mid-write can't leave a half-written file."""
```

```
        tmp = path + ".tmp"
        with open(tmp, "w") as f:
            f.write(text)
            f.flush()
            os.fsync(f.fileno())
        os.replace(tmp, path)


    def _det_plain(det) -> list:
        """A detector candidate dict {'xy': np.array([x,y]), 'score', 'scale'} -> JSON-able
list."""
        xy = det["xy"]
        return [float(xy[0]), float(xy[1]), float(det["score"]), int(det["scale"])]


    def _dets_to_tracker(plain_list):
        """Inverse of _det_plain: rebuild the dicts the tracker expects (xy MUST be a numpy
array,
        the tracker does vector math / np.linalg.norm on it)."""
        import numpy as np
        return [{"xy": np.array([p[0], p[1]], dtype=float), "score": p[2], "scale": p[3]}
                for p in plain_list]


    def write_manifest(cache_dir: str, manifest: dict) -> None:
        _, mpath = _cache_paths(cache_dir)
        _atomic_write_text(mpath, json.dumps(manifest, indent=2))


    def read_manifest(cache_dir: str):
        _, mpath = _cache_paths(cache_dir)
        if not osp.exists(mpath):
            return None
        try:
            with open(mpath) as f:
                return json.load(f)
        except (json.JSONDecodeError, OSError):
            return None


    def manifest_compatible(manifest: dict, *, frames_dir: str, weights: str, sport: str,
                            frames_in: int, frames_total: int, device: str = "cuda") ->
bool:
        """A cache is reusable only if the run that produced it matches this run's inputs.

        ``device`` defaults to ``cuda`` so caches written before device-keying (no
``device``
        field) stay compatible with a CUDA run ? but a cpu/mps run will NOT reuse a cuda
cache
        (detections can differ numerically across devices), and vice-versa.
        """
        if not manifest or manifest.get("version") != CACHE_VERSION:
            return False
        return (
            manifest.get("frames_dir") == osp.abspath(frames_dir)
            and manifest.get("weights") == weights
            and manifest.get("sport") == sport
            and manifest.get("frames_in") == frames_in
            and manifest.get("frames_total") == frames_total
            and manifest.get("device", "cuda") == device
        )


    def reset_cache(cache_dir: str) -> None:
        """Drop any existing cache so the next run starts clean."""
```

```
        det_jsonl, mpath = _cache_paths(cache_dir)
        for p in (det_jsonl, mpath):
            if osp.exists(p):
                os.remove(p)


    def load_and_clean_cache(cache_dir: str):
        """Read the longest *contiguous* (w == line index) valid prefix of det_raw.jsonl,
        rebuild the per-frame detection accumulation, and rewrite the file to exactly that
        prefix so a trailing half-written line from a crash can't corrupt future appends.

        Returns (windows_done, det_by_fid) where det_by_fid maps frame_id -> list of
        [x, y, score, scale] candidates (accumulated across every window the frame is in,
        in window order ? identical to a non-resumed run).
        """
        det_jsonl, _ = _cache_paths(cache_dir)
        det_by_fid = defaultdict(list)
        valid: list = []
        if osp.exists(det_jsonl):
            with open(det_jsonl, "r") as f:
                for line in f:
                    s = line.strip()
                    if not s:
                        continue
                    try:
                        obj = json.loads(s)
                    except json.JSONDecodeError:
                        break                       # truncated trailing line (crash mid-
flush)
                    if obj.get("w") != len(valid):  # non-contiguous -> stop at the gap
                        break
                    valid.append(s)
                    for fid, plain in obj["f"]:
                        det_by_fid[int(fid)].extend([list(p) for p in plain])
            # Guarantee a clean append boundary by rewriting only the good prefix.
            _atomic_write_text(det_jsonl, ("\n".join(valid) + "\n") if valid else "")
        return len(valid), det_by_fid


    def _append_windows(fh, objs) -> None:
        """Append window records as JSONL and durably flush (bounded loss on crash)."""
        for o in objs:
            fh.write(json.dumps(o, separators=(",", ":")) + "\n")
        fh.flush()
        os.fsync(fh.fileno())


    def _atomic_write_trajectory(out_csv: str, rows) -> None:
        os.makedirs(osp.dirname(osp.abspath(out_csv)), exist_ok=True)
        tmp = out_csv + ".tmp"
        with open(tmp, "w", newline="") as f:
            w = csv.writer(f)
            w.writerow(["Frame", "Visibility", "X", "Y"])
            w.writerows(rows)
            f.flush()
            os.fsync(f.fileno())
        os.replace(tmp, out_csv)


    # ---------------------------------------------------------------------- #
    # Frame addressing + windowing (pure; no torch/cv2 ? unit-testable)
    # ---------------------------------------------------------------------- #
    _STREAM_FRAME_FMT = "{:08d}.png"


```

```
    def synthetic_frame_path(index: int) -> str:
        """Stable, index-encoding frame name used by the streaming path.

        It mirrors the on-disk names ``extract_frames`` writes (``{i:08d}.png``) so the
        downstream integer-id parser (``_frame_id``) yields the SAME frame id whether the
        frames came from disk or from the in-memory stream. The streaming image loader
        inverts this name back to an index to fetch the decoded frame.
        """
        return _STREAM_FRAME_FMT.format(int(index))


    def build_windows(frames: list, frames_in: int):
        """Sliding windows of ``frames_in`` consecutive frame *paths* (the unit of GPU work
        + checkpoint). Pure list math, identical for the disk and streaming paths.

        Returns a list of windows, each a list of ``frames_in`` consecutive paths
        (``[frames[i:i+frames_in] for i in range(len(frames) - frames_in + 1)]``). The
        caller wraps each window into the ``{"frames", "annos"}`` sample shape WASB's
        ``ImageDataset`` expects; keeping that out of here stays torch-free for unit tests.
        """
        return [frames[i:i + frames_in] for i in range(len(frames) - frames_in + 1)]


    class SequentialFrameStore:
        """In-memory sliding buffer over a forward-only frame reader, sized for the
        detector's sliding-window access pattern (peak O(window) frames, not O(video)).

        The detector DataLoader runs with ``shuffle=False`` and (in streaming mode)
        ``num_workers=0``. ``ImageDataset.__getitem__(i)`` reads the frames of window ``i``
        in order ? ``i, i+1, ..., i+window-1`` ? and samples are requested ``i = start,
        start+1, ...``. So the global read sequence dips back by ``window-1`` at each window
        boundary (``0,1,2, 1,2,3, 2,3,4, ...`` for ``window=3``); the highest index seen so
        far never decreases. We keep only frames at or above ``max_seen - (window-1)`` (the
        earliest index any not-yet-finished window can still ask for) and decode each source
        frame exactly once via the forward-only ``reader(index)``.
        """

        def __init__(self, reader, total: int, window: int = 1, start_index: int = 0):
            self._reader = reader
            self._total = int(total)
            self._window = max(1, int(window))
            self._buf: dict = {}
            # On a resumed run the detector's first needed frame is `start_index`; begin
            # pulling there so the reader can skip (seek past) the already-cached prefix
            # instead of decoding 0..start_index-1 just to throw them away.
            self._next = int(start_index)       # next index not yet pulled from the reader
            self._max_seen = int(start_index) - 1
            self._floor = int(start_index)      # lowest index we still keep (never evict
below)

        def get(self, index: int):
            index = int(index)
            if index < self._floor:
                raise KeyError(
                    f"frame {index} already evicted (below floor {self._floor}); the access
"
                    f"pattern must stay within a window of the highest frame seen")
            if index >= self._total:
                raise KeyError(f"frame {index} out of range (total {self._total})")
            # Pull forward from the reader until the requested index is buffered.
            while self._next <= index:
                self._buf[self._next] = self._reader(self._next)
                self._next += 1
            if index > self._max_seen:
                self._max_seen = index
```

```
            # Evict frames older than the earliest a still-running window could re-request:
            # once we've seen index `m`, no future window starts before `m - (window-1)`.
            new_floor = max(self._floor, self._max_seen - (self._window - 1))
            for k in range(self._floor, new_floor):
                self._buf.pop(k, None)
            self._floor = new_floor
            return self._buf[index]


    def _index_from_synthetic_path(path: str) -> int:
        """Invert ``synthetic_frame_path``: a frame path -> its integer source index.

        Reuses ``_frame_id`` (digits-only parse) so the index and the cached frame-id stay
        in lockstep with the on-disk naming scheme.
        """
        return _frame_id(path)


    def _bgr_to_pil_rgb(frame_bgr):
        """Convert an in-memory BGR uint8 frame (as ``cv2.VideoCapture`` yields) to the
EXACT
        PIL RGB image WASB's disk path produces.

        The disk path is ``frame_bgr -> cv2.imwrite(PNG) ->
Image.open(PNG).convert('RGB')``;
        because PNG is lossless that round-trip equals ``cv2.cvtColor(frame_bgr,
COLOR_BGR2RGB)``
        bit-for-bit (verified empirically, max|diff|=0). Returning that here makes the
streamed
        frame indistinguishable from the on-disk one to everything downstream.
        """
        import cv2
        from PIL import Image
        return Image.fromarray(cv2.cvtColor(frame_bgr, cv2.COLOR_BGR2RGB))


    def _make_streamed_read_image(store, real_read_image):
        """Build the ``read_image`` replacement used during a streaming detector pass.

        A synthetic frame path -> the in-memory decoded frame for its index (as a PIL RGB
image,
        matching ``read_image``'s real return type); any path that doesn't parse to an index
falls
        through to ``real_read_image``. Pure ? imports no WASB module ? so it is unit-
testable
        without the WSL-only ``dataloaders`` package.
        """
        def _read_image(path, *args, **kwargs):
            idx = _index_from_synthetic_path(path)
            if idx < 0:
                return real_read_image(path, *args, **kwargs)
            return _bgr_to_pil_rgb(store.get(idx))
        return _read_image


    @contextmanager
    def _streaming_read_image_patch(frame_loader, total: int, window: int = 1, start_index:
int = 0):
        """Scope a shim over WASB's ``read_image`` so ``ImageDataset`` is fed in-memory
decoded
        frames during the detector pass, byte-for-byte as it would over real PNGs.

        WASB's ``ImageDataset.__getitem__`` reads each frame via ``read_image(path)`` ? NOT
        ``cv2.imread``. ``read_image`` (``utils.utils``) does
``Image.open(path).convert('RGB')``,
```

```
341         returning a PIL **RGB** image, after an ``osp.exists(path)`` guard. We therefore
feed it a
342         PIL RGB image built from the streamed BGR frame (see ``_bgr_to_pil_rgb``), which is
343         byte-identical to the disk round-trip, and the shim never touches the filesystem so
the
344         ``osp.exists`` guard is sidestepped for synthetic stream paths.
345
346         We patch ``dataloaders.dataset_loader.read_image`` ? the binding the consumer
actually
347         calls (``dataset_loader`` does ``from utils import read_image``, so the name lives
in that
348         module's namespace; patching ``utils.read_image`` would NOT rebind the already-
imported
349         reference).
350
351         ``frame_loader(index) -> BGR uint8 ndarray`` is a forward-only sequential decoder
wrapped
352         in a ``SequentialFrameStore`` (sliding buffer sized to ``window`` = ``frames_in``);
on a
353         resume we start at ``start_index`` so the decoder seeks past already-cached windows.
The
354         original reader is restored on exit.
355         """
356         from dataloaders import dataset_loader as _dl
357         store = SequentialFrameStore(frame_loader, total, window=window,
start_index=start_index)
358         real_read_image = _dl.read_image
359         # Rebind read_image in the consuming module for the duration of the detector pass;
360         # restore on exit. (Plain assignment, not setattr-with-constant, to stay ruff
B010-clean.)
361         _dl.read_image = _make_streamed_read_image(store, real_read_image)  # type:
ignore[assignment]
362         try:
363             yield store
364         finally:
365             _dl.read_image = real_read_image  # type: ignore[assignment]
366
367
368     # ---------------------------------------------------------------------------- #
369     # Inference
370     # ---------------------------------------------------------------------------- #
371     def run(frames, weights: str, out_csv: str, sport: str = "badminton",
372             limit: int = 0, batch_size: int = 8, num_workers: int = 4,
373             log_every_batches: int = 50, cache_dir: "str | None" = None,
374             flush_every: int = 1000, fresh: bool = False,
375             frame_loader=None, source_key: "str | None" = None,
376             device: str = "cuda") -> str:
377         """Run WASB detection+tracking over an ordered frame source -> trajectory CSV.
378
379         Two equivalent ways to supply pixels (output is bit-identical between them):
380
381         * **frames-on-disk** (default): ``frames`` is a directory path string; frames are
382           globbed (``*.png``/``*.jpg``) and ``ImageDataset`` reads each file via
``cv2.imread``.
383         * **streaming** (fast path): ``frames`` is a pre-built ordered list of (synthetic)
384           frame paths and ``frame_loader(index) -> BGR uint8 ndarray`` supplies the decoded
385           pixels in-memory. We scope a shim over WASB's ``read_image`` (the actual per-frame
386           reader, which returns a PIL RGB image) over the DataLoader pass so every other
line of
387           ``ImageDataset.__getitem__`` runs UNCHANGED ? the tensor the detector sees is
identical
388           to the disk round-trip (``cv2.imwrite`` PNG -> ``Image.open().convert('RGB')`` ==
389           ``cvtColor(bgr, BGR2RGB)``, verified byte-identical). Streaming forces
``num_workers=0``
390           (the in-process shim + buffer can't cross worker processes).
```

```
391
392         ``source_key`` overrides the cache-keying identity (defaults to the abspath of the
393         frames dir); the streaming path passes a stable ``video:<abspath>`` key so a resume
394         after reboot still matches.
395         """
396         import time
397         import torch
398         from torch.utils.data import DataLoader
399         from detectors import build_detector
400         from trackers import build_tracker
401         from dataloaders import build_img_transforms, build_seq_transforms
402         from dataloaders.dataset_loader import ImageDataset
403         from utils import Center
404
405         streaming = frame_loader is not None
406
407         cfg = _build_cfg(weights, sport, device)
408         frames_in = int(cfg["model"]["frames_in"])
409         input_wh = (int(cfg["model"]["inp_width"]), int(cfg["model"]["inp_height"]))
410         output_wh = (int(cfg["model"]["out_width"]), int(cfg["model"]["out_height"]))
411
412         if streaming:
413             # `frames` is already the ordered list of (synthetic) frame paths.
414             if not isinstance(frames, (list, tuple)):
415                 raise SystemExit("streaming mode requires `frames` to be a list of frame
paths")
416             frames = list(frames)
417             frames_dir = source_key or "stream"
418         else:
419             frames_dir = frames
420             frames = sorted(glob.glob(osp.join(frames_dir, "*.png")) +
glob.glob(osp.join(frames_dir, "*.jpg")))
421         if limit:
422             frames = frames[:limit]
423         if len(frames) < frames_in:
424             where = frames_dir if not streaming else "video stream"
425             raise SystemExit(f"need >= {frames_in} frames, found {len(frames)} in {where}")
426
427         # Sliding windows of `frames_in` consecutive frames (the unit of GPU work +
checkpoint).
428         samples = []
429         for window in build_windows(frames, frames_in):
430             annos = [{"frame_path": p, "center": Center(is_visible=False, x=-1.0, y=-1.0)}
for p in window]
431             samples.append({"frames": window, "annos": annos})
432         windows_total = len(samples)
433
434         # ---- cache setup / resume decision ---------------------------------- #
435         if cache_dir is None:
436             cache_dir = default_cache_dir(out_csv)
437         os.makedirs(cache_dir, exist_ok=True)
438         det_jsonl, _ = _cache_paths(cache_dir)
439         cache_key = source_key if source_key is not None else osp.abspath(frames_dir)
440         manifest = None if fresh else read_manifest(cache_dir)
441         resume = manifest_compatible(
442             manifest or {}, frames_dir=cache_key, weights=weights, sport=sport,
443             frames_in=frames_in, frames_total=len(frames), device=device,
444         )
445         if resume:
446             windows_done, det_by_fid = load_and_clean_cache(cache_dir)
447             logger.info(f"resuming from cache: {windows_done}/{windows_total} windows
already detected")
448         else:
449             if manifest is not None:
450                 logger.info("cache incompatible with this run -> starting fresh")
```

```
            reset_cache(cache_dir)
            windows_done, det_by_fid = 0, defaultdict(list)

        base_manifest = {
            "version": CACHE_VERSION,
            "frames_dir": cache_key,
            "weights": weights,
            "sport": sport,
            "frames_in": frames_in,
            "frames_total": len(frames),
            "device": device,
            "windows_total": windows_total,
            "windows_done": windows_done,
        }
        write_manifest(cache_dir, base_manifest)

        # ---- detector pass over the un-cached windows ---------------------- #
        remaining = samples[windows_done:]
        if remaining:
            _, transform_test = build_img_transforms(cfg)
            try:
                _, seq_transform_test = build_seq_transforms(cfg)
            except Exception:
                seq_transform_test = None
            ds = ImageDataset(cfg, remaining, input_wh, output_wh,
                              transform=transform_test, seq_transform=seq_transform_test,
is_train=False)
            # Streaming forces single-process loading: the in-memory frame store + the
            # cv2.imread shim live in THIS process and can't be pickled to DataLoader
workers.
            loader_workers = 0 if streaming else num_workers
            loader = DataLoader(ds, batch_size=batch_size, shuffle=False,
num_workers=loader_workers)
            detector = build_detector(cfg)

            from contextlib import ExitStack

            t0 = time.time()
            done = windows_done
            win_idx = windows_done
            log_every = max(1, log_every_batches)
            flush_n = max(1, flush_every)
            pending = []
            logger.info(f"detecting on {len(remaining)} remaining windows "
                        f"(total {windows_total}, batch={batch_size},
workers={loader_workers}, "
                        f"source={'video-stream' if streaming else 'frames-dir'})...")
            det_f = open(det_jsonl, "a")
            try:
                with ExitStack() as stack:
                    stack.enter_context(torch.no_grad())
                    # In streaming mode, route ImageDataset's per-frame `cv2.imread(path)`
to the
                    # in-memory decoded frame for that synthetic path; every other line of
                    # __getitem__ runs unchanged so the tensor matches the PNG round-trip
exactly.
                    # The detector's first needed frame is `windows_done` (window i starts
at
                    # frame i), so prime the store there for a resume.
                    if streaming:
                        stack.enter_context(
                            _streaming_read_image_patch(frame_loader, len(frames),
                                                        window=frames_in,
start_index=windows_done))
                    for bi, batch in enumerate(loader):
```

```
                        imgs, _hms, trans = batch[0], batch[1], batch[2]
                        img_paths = [list(t) for t in batch[-1]]   # [frame_in][batch] ->
path
                        batch_results, _ = detector.run_tensor(imgs, trans)
                        nb = imgs.shape[0]
                        for ib in range(nb):
                            frames_payload = []
                            for ie in sorted(batch_results[ib].keys()):
                                p = img_paths[ie][ib]
                                fid = _frame_id(p)
                                plain = [_det_plain(d) for d in batch_results[ib][ie]]
                                frames_payload.append([fid, plain])
                                det_by_fid[fid].extend(plain)
                            pending.append({"w": win_idx, "f": frames_payload})
                            win_idx += 1
                        done += nb
                        if len(pending) >= flush_n or done >= windows_total:
                            _append_windows(det_f, pending)
                            pending = []
                            base_manifest["windows_done"] = done
                            write_manifest(cache_dir, base_manifest)
                        if (bi + 1) % log_every == 0 or done >= windows_total:
                            el = time.time() - t0
                            new_done = done - windows_done
                            rate = new_done / el if el > 0 else 0.0
                            eta = (windows_total - done) / rate if rate > 0 else 0.0
                            logger.info(f"{done}/{windows_total} "
                                        f"({100 * done // max(1, windows_total)}%) |
{rate:.1f} win/s | "
                                        f"elapsed {el:.0f}s | ETA {eta:.0f}s")
                    if pending:
                        _append_windows(det_f, pending)
                        base_manifest["windows_done"] = done
                        write_manifest(cache_dir, base_manifest)
            finally:
                det_f.close()
            logger.info(f"detection done in {time.time() - t0:.0f}s; running tracker...")
        else:
            logger.info("detector cache already complete -> tracker-only re-run")

        # ---- tracker re-run over the full ordered cache (cheap, deterministic) ---- #
        tracker = build_tracker(cfg)
        tracker.refresh()
        rows = []
        for fid in sorted(det_by_fid.keys()):
            r = tracker.update(_dets_to_tracker(det_by_fid[fid]))
            rows.append((fid, 1 if r.get("visi") else 0, r.get("x", -1), r.get("y", -1)))

        _atomic_write_trajectory(out_csv, rows)
        visible = sum(1 for _, v, *_ in rows if v)
        logger.info(f"wrote {len(rows)} rows -> {out_csv}")
        logger.info(f"visible-shuttle frames: {visible}/{len(rows)}")
        return out_csv


    def extract_frames(video_path: str, frames_dir: str) -> str:
        """Decode a video to PNG frames named by source frame index (00000000.png ...).

        Idempotent: if the dir already holds the expected number of frames (matching the
        container's reported frame count) we skip re-decoding ? re-extraction after a
        reboot is wasteful. NOTE: cv2 PNG extraction is slow; for big clips prefer
        extracting with ffmpeg on the host and passing --frames_dir.
        """
        import cv2
        os.makedirs(frames_dir, exist_ok=True)
```

```
        cap = cv2.VideoCapture(video_path)
        if not cap.isOpened():
            raise SystemExit(f"cannot open video: {video_path}")
        expected = int(cap.get(cv2.CAP_PROP_FRAME_COUNT) or 0)
        existing = len(glob.glob(osp.join(frames_dir, "*.png")))
        if expected > 0 and existing >= expected:
            cap.release()
            logger.info(f"reusing {existing} already-extracted frames -> {frames_dir}")
            return frames_dir
        i = 0
        while True:
            ok, frame = cap.read()
            if not ok:
                break
            cv2.imwrite(osp.join(frames_dir, f"{i:08d}.png"), frame)
            i += 1
        cap.release()
        logger.info(f"extracted {i} frames -> {frames_dir}")
        return frames_dir


    def _probe_frame_count(video_path: str) -> int:
        """Container-reported frame count via cv2 (0 if unknown). Used to size the
stream."""
        import cv2
        cap = cv2.VideoCapture(video_path)
        if not cap.isOpened():
            raise SystemExit(f"cannot open video: {video_path}")
        n = int(cap.get(cv2.CAP_PROP_FRAME_COUNT) or 0)
        cap.release()
        return n


    def make_video_frame_loader(video_path: str):
        """Return ``(frame_loader, count_hint, release)`` for output-preserving streaming.

        ``frame_loader(index) -> BGR uint8 ndarray`` decodes the video forward-only with one
        long-lived ``cv2.VideoCapture``. It MUST be called with non-decreasing indices (the
        detector's sliding-window order); it reads sequentially and only uses a frame-seek
to
        skip forward when a *resume* starts past the current position. Each ``cap.read()``
        returns exactly the BGR uint8 array that ``cv2.imwrite``/``cv2.imread`` of a PNG
would
        yield (PNG is lossless), so the detector sees identical pixels to the disk path.

        Returns the count hint from the container so the caller can build the frame list;
        ``release`` closes the capture.
        """
        import cv2
        cap = cv2.VideoCapture(video_path)
        if not cap.isOpened():
            raise SystemExit(f"cannot open video: {video_path}")
        count_hint = int(cap.get(cv2.CAP_PROP_FRAME_COUNT) or 0)
        state = {"pos": 0}  # next index cap.read() will return

        def frame_loader(index: int):
            index = int(index)
            if index < state["pos"]:
                raise RuntimeError(
                    f"streaming decode is forward-only; got index {index} after
{state['pos']}")
            if index > state["pos"]:
                # Forward skip only happens when resuming past cached windows. Seek once.
                cap.set(cv2.CAP_PROP_POS_FRAMES, index)
                state["pos"] = index
```

```
            ok, frame = cap.read()
            if not ok:
                raise RuntimeError(f"failed to decode frame {index} from {video_path}")
            state["pos"] += 1
            return frame

        def release():
            cap.release()

        return frame_loader, count_hint, release


    def run_video_streaming(video_path: str, weights: str, out_csv: str, sport: str =
"badminton",
                            limit: int = 0, batch_size: int = 8,
                            log_every_batches: int = 50, cache_dir: "str | None" = None,
                            flush_every: int = 1000, fresh: bool = False,
                            device: str = "cuda") -> str:
        """Fast path: decode ``video_path`` on the fly and feed frames straight to the
        detector, WITHOUT extracting every frame to a PNG first (the disk round-trip that
        dominates a long clip). Output is bit-identical to the frames-on-disk path; see
        ``run``'s docstring for why.
        """
        frame_loader, count_hint, release = make_video_frame_loader(video_path)
        try:
            if count_hint <= 0:
                raise SystemExit(
                    f"could not determine frame count for {video_path}; cannot stream "
                    f"(fall back to frame extraction with --frames_dir)")
            frame_list = [synthetic_frame_path(i) for i in range(count_hint)]
            source_key = "video:" + osp.abspath(video_path)
            return run(frame_list, weights, out_csv, sport, limit,
                       batch_size=batch_size, num_workers=0,
                       log_every_batches=log_every_batches, cache_dir=cache_dir,
                       flush_every=flush_every, fresh=fresh,
                       frame_loader=frame_loader, source_key=source_key, device=device)
        finally:
            release()


    def main():
        ap = argparse.ArgumentParser()
        ap.add_argument("--frames_dir", help="folder of pre-extracted frames")
        ap.add_argument("--video", help="video file; frames are extracted internally (in
WSL)")
        ap.add_argument("--frames_out_dir", help="where to extract frames when --video is
used")
        ap.add_argument("--stream-video", action="store_true",
                        help="FAST PATH: decode --video on the fly and feed frames straight
to "
                             "the detector (no PNG extraction). Output-equivalent to the
disk path.")
        ap.add_argument("--weights", required=True)
        ap.add_argument("--out", required=True)
        ap.add_argument("--sport", default="badminton")
        ap.add_argument("--device", default="cuda", choices=["cuda", "mps", "cpu"],
                        help="torch device for the detector (default cuda = the WSL parity
path)")
        ap.add_argument("--limit", type=int, default=0, help="cap #frames (for quick
tests)")
        ap.add_argument("--batch-size", type=int, default=8)
        ap.add_argument("--num-workers", type=int, default=4)
        ap.add_argument("--log-every-batches", type=int, default=50, help="progress log
cadence")
        ap.add_argument("--cache-dir", default=None,
```

```
689                         help="resumable detector cache dir (default: <out>_wasbcache next to
--out)")
690         ap.add_argument("--flush-every", type=int, default=1000,
691                         help="fsync the detector cache every N windows (bounds reboot
loss)")
692         ap.add_argument("--fresh", action="store_true", help="ignore any existing cache and
start over")
693         args = ap.parse_args()
694
695         if args.stream_video:
696             if not args.video:
697                 raise SystemExit("--stream-video requires --video")
698             run_video_streaming(args.video, args.weights, args.out, args.sport, args.limit,
699                                 batch_size=args.batch_size,
700                                 log_every_batches=args.log_every_batches,
cache_dir=args.cache_dir,
701                                 flush_every=args.flush_every, fresh=args.fresh,
device=args.device)
702             return
703
704         frames_dir = args.frames_dir
705         if args.video:
706             frames_dir = extract_frames(args.video, args.frames_out_dir or (args.video +
"_frames"))
707         if not frames_dir:
708             raise SystemExit("provide --frames_dir or --video")
709         run(frames_dir, args.weights, args.out, args.sport, args.limit,
710             batch_size=args.batch_size, num_workers=args.num_workers,
711             log_every_batches=args.log_every_batches, cache_dir=args.cache_dir,
712             flush_every=args.flush_every, fresh=args.fresh, device=args.device)
713
714
715     if __name__ == "__main__":
716         main()
717
```

6. assistant (2026-06-20T08:57:19.839Z)

Now let me read the other changed files and the base/tracknet runner to compare contracts precisely.

7. user (2026-06-20T08:57:20.818Z)

```
1       """Abstract base for detector runners (TrackNet, WASB, future native, etc.).
2
3       This completes the Detector Runner Abstraction (low-regret item 6 continuation / item 1
4       of the post-observability re-prioritization) to de-risk the WSL/conda dependency for
5       the high-accuracy shuttle trajectory substrate used by tracknet_hybrid / wasb_hybrid.
6
7       The ABC lets us:
8       - Keep the current WSL adapters working unchanged for Windows users.
9       - Swap in (or A/B test) a Linux-native implementation later with zero changes to
10        the hybrid segmenters or the model-agnostic windowing brain.
11      - Provide a single place to evolve the healthcheck / error contract and thread
12        detector readiness into per-video reports.
13
14      Contract (deliberately matches what the concrete runners + hybrids + tests already did):
15      - run_predict(video_win_path: str, output_win_dir: str) -> Optional[str]
16        Run detector, return the Windows-side path to the produced trajectory CSV
17        (e.g. ".../clip_predict.csv" or ".../clip_wasb.csv"), or None on any failure.
18        The caller (hybrid) is responsible for output dir; impls may stage the video
19        into WSL fs for perf.
20
21      - healthcheck() -> tuple[bool, str]
22        Return (ok, message). Never raises. "ok" decides whether to proceed to run_predict.
```

```
        Message is logged (and can be surfaced to reports or UX in future).
        Intended to be cheap (env existence, import, GPU visibility, weights present).

    Existing behavior is 100% preserved. The only additions are the inheritance and
    the explicit common type for future implementers.

    How to add a new runner (for docs / future contributors):
    1. Subclass DetectorRunner in a new or existing module under pipeline/detectors/.
    2. Implement run_predict and healthcheck (return (bool, str) tuple).
    3. If it produces the same CSV schema (Frame,Visibility,X,Y), reuse or re-export
       TrackNetRunner.parse_trajectory_csv and .trajectory_to_action_windows (they are
       deliberately static and model-agnostic). If the new detector needs different
       windowing, it can implement its own or we generalize later.
    4. Lazy-import heavy native deps inside the methods (so importing the segmenter
       registry never pulls torch/tf on a machine that doesn't have them).
    5. Update the two hybrid segmenters (or introduce a detector= choice) to accept
       DetectorRunner instances (they already do the right thing via duck-typing + the
       type hints we add here).
    6. Add unit tests that exercise the contract (isinstance, healthcheck shape, run
       returns str|None) using mocks for the external bits ? see test_tracknet_runner.py.
    7. No change to eval/calibration code paths (they consume the CSV + the static
       windowing helpers directly).

    Optional Linux-native path (the main de-risk goal):
    A LinuxNativeRunner (or ONNXRunner) would:
    - Skip all wsl / _stage / conda activate.
    - Load weights locally (torch or onnx) or call a native binary.
    - Write the exact same CSV format so trajectory_to_action_windows works verbatim.
    - healthcheck can just check "weights file exists and torch importable + CUDA?".
    This removes the WSL requirement for Linux users / future SaaS workers while
    keeping the exact same candidate quality characteristics (or better, once native
    weights are available).

    Error surfacing into reports:
    The healthcheck message is already logged by the hybrids on failure. The reporting
    harvest sees the outcome as "0 candidates" + detector name. When we wire richer
    segmenter Results (or pass detector_health into emit_indexer_report context), the
    (ok, msg) can be recorded explicitly under segmentation.details or a new detector
    stage. The ABC makes that future change local to one place.

    See also:
    - backend/pipeline/segmenters/{tracknet_hybrid,wasb_hybrid}.py (the two callers)
    - tests/test_tracknet_runner.py (contract + pure logic tests; hybrids mock the runner)
    - docs/TRACKNET_WSL_SETUP.md and CODE_MAP.md (2.2 Detectors section)
    """

    import logging
    import shlex
    import subprocess
    from abc import ABC, abstractmethod
    from typing import Any, Callable, List, Optional, Tuple

    logger = logging.getLogger(__name__)


    def wsl_tilde_quote(path: str) -> str:
        """shlex.quote that still lets bash expand a leading ``~/``.

        ``shlex.quote("~/clips/x.mp4")`` single-quotes the whole path, so bash
        treats ``~`` literally and ``cp``/``mkdir`` fail with "No such file or
        directory" ? this broke live WSL staging for the default ``~/clips``
        stage dir (found by the 2026-06-11 wasb_hybrid smoke run; the BACKLOG
        "shlex-quote breaks ~ expansion" item). Quoting only the part after
        ``~/`` keeps expansion AND protects spaces/specials in the remainder.
        """
```

```
        if path == "~":
            return path
        if path.startswith("~/"):
            return "~/" + shlex.quote(path[2:])
        return shlex.quote(path)


    class WslCommandMixin:
        """Shared ``wsl -d <distro> bash -c 'source <conda_sh> && <cmd>'`` invocation.

        Lives beside (not on) DetectorRunner: a future Linux-native runner must not
        inherit WSL plumbing. Requires ``self.cfg`` with ``distro``, ``conda_sh``
        and ``timeout_sec`` attributes (TrackNetConfig and WasbConfig both qualify).
        ``timeout_sec == 0`` maps to None = wait forever (configs default 1800s).
        """

        cfg: Any

        def _wsl_bash(self, inner_cmd: str) -> subprocess.CompletedProcess:
            full = f"source {self.cfg.conda_sh} && {inner_cmd}"
            argv = ["wsl", "-d", self.cfg.distro, "bash", "-c", full]
            logger.debug("WSL exec: %s", full)
            return subprocess.run(argv, capture_output=True, text=True,
                                  timeout=(self.cfg.timeout_sec or None))

        # ---- cache hygiene helpers (shared by both WSL runners) ------------------
        def _wsl_free_gb(self, wsl_path: str) -> Optional[float]:
            """Best-effort free space (GB) on the WSL filesystem holding ``wsl_path``.

            Returns None if it can't be determined (never raises). Used as a pre-run
            disk guard so a long extraction doesn't fill the disk mid-flight.
            """
            try:
                res = self._wsl_bash(f"df -P -B1 {wsl_tilde_quote(wsl_path)} | awk
'NR==2{{print $4}}'")
            except (subprocess.TimeoutExpired, OSError):
                return None
            try:
                return int((res.stdout or "").strip()) / (1024 ** 3)
            except (ValueError, AttributeError):
                return None

        def _wsl_rm_rf(self, wsl_paths: List[str]) -> None:
            """Best-effort recursive delete of WSL paths (post-success cleanup; never
raises)."""
            paths = [p for p in wsl_paths if p]
            if not paths:
                return
            quoted = " ".join(wsl_tilde_quote(p) for p in paths)
            try:
                self._wsl_bash(f"rm -rf {quoted}")
            except (subprocess.TimeoutExpired, OSError) as e:
                logger.warning("WSL cache cleanup failed (non-fatal): %s", e)

        def wsl_source_unreadable_reason(self, src_mnt: str) -> Optional[str]:
            """Actionable message if ``src_mnt`` is NOT readable from inside WSL, else None.

            Google Drive (``G:``/Drive File Stream), UNC and many network/virtual
            filesystems are invisible to WSL's ``/mnt`` ? staging a video from such a path
            fails with a cryptic ``cp: cannot stat`` and the whole run silently produces no
            trajectory (i.e. "0 rallies", a wasted run that looks like a quality result).
            A cheap ``test -r`` preflight turns that into one clear, fixable error BEFORE
any
            frame extraction. Best-effort: a flaky/timed-out probe returns None so the real
            ``cp`` still gets its chance (we never block a genuinely-readable source).
```

```
            """
            try:
                res = self._wsl_bash(f"test -r {shlex.quote(src_mnt)} && echo READABLE")
            except (subprocess.TimeoutExpired, OSError):
                return None
            if "READABLE" in (res.stdout or ""):
                return None
            return (
                f"video source is not readable from WSL at '{src_mnt}'. Google Drive, UNC, "
                f"and other network/virtual filesystems are invisible to WSL's /mnt ? copy "
                f"the video to a local disk (e.g. C:\\ or the WSL filesystem) and retry."
            )


    class DetectorRunner(ABC):
        """Common interface for shuttle/ball trajectory detectors (WSL or otherwise)."""

        @abstractmethod
        def run_predict(self, video_win_path: str, output_win_dir: str,
                        video_id: Optional[str] = None) -> Optional[str]:
            """Run inference and return path to trajectory CSV (or None on failure).

            ``video_id`` (the file MD5) namespaces all staged/cached/output artifacts so two
            videos that happen to share a filename cannot reuse each other's frames or CSV.
            It is optional for back-compat; callers in the live pipeline always pass it.
            """
            pass

        @abstractmethod
        def healthcheck(self) -> Tuple[bool, str]:
            """Quick check if the runner (env, weights, etc.) is usable.

            Returns (ok, message). Never raises. Cheap to call before a long run.
            """
            pass

        # De facto trajectory contract: the trajectory hybrids call these
        # unconditionally after run_predict. Concrete runners reuse/re-export the
        # model-agnostic TrackNetRunner staticmethods (same CSV schema + windowing);
        # a runner with custom windowing supplies its own with the same shapes.
        parse_trajectory_csv: Callable[..., Any]
        trajectory_to_action_windows: Callable[..., Any]
```

8. user (2026-06-20T08:57:21.817Z)

```
    """TrackNet shuttle-trajectory runner (WSL2 subprocess adapter).

    This bridges the Windows-side indexer to a TensorFlow TrackNetV4 install living
    inside WSL2 (ADR-001 in docs/DECISIONS.md). The flow is:

        1. Translate the Windows video path to its WSL-visible form (and optionally
           stage it into the WSL filesystem to avoid slow /mnt/c per-frame reads).
        2. Invoke ``src/predict.py`` inside the TrackNetV4 conda env via ``wsl``.
        3. Read back the per-frame ``Frame,Visibility,X,Y`` CSV it writes.
        4. Convert that trajectory into high-motion "action windows" ? the same
           candidate-rally shape HybridYoloSegmenter produces ? so the rest of the
           pipeline (AI handoff, DB population) is unchanged.

    Nothing here imports TensorFlow; all model code stays in WSL.
    """

    import os
    import csv
    import shlex
```

```
    import logging
    import subprocess
    from dataclasses import dataclass
    from typing import List, Dict, Any, Optional, Tuple

    from backend.pipeline.detectors.base import DetectorRunner, WslCommandMixin,
wsl_tilde_quote

    logger = logging.getLogger(__name__)


    @dataclass
    class TrackNetConfig:
        """Resolved settings for a TrackNet WSL run (built from config.json ->
indexing.tracknet)."""
        distro: str = "Ubuntu"
        repo_dir: str = "~/models/TrackNetV4"          # WSL path to the cloned repo
        conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
        conda_env: str = "TrackNetV4"
        weights_path: str = ""                          # WSL path to the .h5/.keras weights
(REQUIRED)
        queue_length: int = 5
        stage_in_wsl: bool = True                       # copy video into WSL fs first
(perf)
        wsl_stage_dir: str = "~/clips"                  # where staged videos/outputs live
in WSL
        timeout_sec: int = 1800                         # 30 min; kills a hung call (0 = no
timeout)
        keep_frames: bool = False                       # retain staged video after success
(debug only)
        min_free_gb: float = 0.0                        # warn if WSL free space below this
(0 = off)

        @classmethod
        def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "TrackNetConfig":
            tn = (idx_cfg or {}).get("tracknet", {}) or {}
            return cls(
                distro=tn.get("wsl_distro", "Ubuntu"),
                repo_dir=tn.get("repo_dir", "~/models/TrackNetV4"),
                conda_sh=tn.get("conda_sh", "~/miniconda3/etc/profile.d/conda.sh"),
                conda_env=tn.get("conda_env", "TrackNetV4"),
                weights_path=tn.get("weights_path", ""),
                queue_length=int(tn.get("queue_length", 5)),
                stage_in_wsl=bool(tn.get("stage_in_wsl", True)),
                wsl_stage_dir=tn.get("wsl_stage_dir", "~/clips"),
                timeout_sec=int(tn.get("timeout_sec", 1800)),
                keep_frames=bool(tn.get("keep_frames", False)),
                min_free_gb=float(tn.get("min_free_gb", 0.0)),
            )


    def to_wsl_mnt_path(win_path: str) -> str:
        """Translate a Windows path (C:\\Users\\x) to its WSL /mnt form (/mnt/c/Users/x).

        Already-POSIX paths (starting with / or ~) are returned unchanged so the
        same helper is safe to call on either kind of path.
        """
        p = str(win_path)
        if p.startswith("/") or p.startswith("~"):
            return p
        p = p.replace("\\", "/")
        if len(p) >= 2 and p[1] == ":":
            drive = p[0].lower()
            return f"/mnt/{drive}{p[2:]}"
        return p
```

```

    @dataclass
    class TrajectoryPoint:
        frame: int
        visible: bool
        x: float
        y: float


    class TrackNetRunner(WslCommandMixin, DetectorRunner):
        """Runs TrackNet predict.py in WSL and turns the output CSV into action windows.

        Implements the common DetectorRunner contract so a future Linux-native or
        alternative trajectory runner can be substituted without touching the hybrids.
        """

        def __init__(self, cfg: TrackNetConfig):
            self.cfg = cfg

        def healthcheck(self) -> Tuple[bool, str]:
            """Verify the WSL env is usable: conda env exists, TF imports, GPU visible.

            Returns (ok, message). Cheap to call before a long run.
            """
            cmd = (
                f"conda activate {self.cfg.conda_env} && "
                f"python -c \"import tensorflow as tf; "
                f"print('TF', tf.__version__, 'GPUs',
len(tf.config.list_physical_devices('GPU')))\""
            )
            try:
                res = self._wsl_bash(cmd)
            except FileNotFoundError:
                return False, "`wsl` not found ? is this running on Windows with WSL2
installed?"
            except subprocess.TimeoutExpired:
                return False, "WSL healthcheck timed out."
            out = (res.stdout or "").strip()
            if res.returncode != 0:
                return False, f"WSL env check failed: {(res.stderr or out).strip()}"
            return True, out

        # ------------------------------------------------------------------ inference

        def run_predict(self, video_win_path: str, output_win_dir: str,
                        video_id: Optional[str] = None) -> Optional[str]:
            """Run predict.py on a video. Returns the Windows path to the produced CSV, or
None.

            ``output_win_dir`` is a Windows directory (under the repo's output/) that
            WSL writes into via its /mnt view, so the Windows process can read the CSV
            back with no copy. ``video_id`` (file MD5) namespaces the staged video ? and
            therefore predict.py's ``<stem>_predict.csv`` output ? so two videos that share
            a filename cannot collide on the output CSV.
            """
            if not self.cfg.weights_path:
                logger.error(
                    "TrackNet weights_path is not set. Set indexing.tracknet.weights_path "
                    "in config.json to the WSL path of your trained TrackNetV4 weights."
                )
                return None

            # Resolve relative dirs BEFORE the /mnt translation ? to_wsl_mnt_path passes
            # relative paths through unchanged, so WSL would resolve them against the
```

```
            # predict.py cwd instead of the repo (same bug class as wasb_runner).
            output_win_dir = os.path.abspath(output_win_dir)
            os.makedirs(output_win_dir, exist_ok=True)
            # Normalize separators so basename/splitext work when tests (or collector)
            # pass Windows-style paths on a Linux host.
            video_win_path = str(video_win_path).replace("\\", "/")
            base = os.path.basename(video_win_path)
            stem, ext = os.path.splitext(base)

            # predict.py only accepts .avi/.mp4 and names the CSV "<stem>_predict.csv".
            if ext.lower() not in (".mp4", ".avi"):
                logger.error("TrackNet predict.py only supports .mp4/.avi (got %s)", ext)
                return None

            wsl_out = to_wsl_mnt_path(output_win_dir)
            # Namespace by video_id so two same-filename videos don't collide on the CSV.
            # predict.py derives its output name from the (staged) video stem, so staging
            # under the namespaced name propagates into "<key>_predict.csv".
            key = f"{stem}__{video_id[:12]}" if video_id else stem
            expected_csv_win = os.path.join(output_win_dir, f"{key}_predict.csv")

            # Resolve the video path WSL will read.
            if self.cfg.stage_in_wsl:
                staged = self._stage_video(video_win_path, f"{key}{ext}")
                if staged is None:
                    return None
                wsl_video = staged
            else:
                # No staging: predict.py reads /mnt directly and writes
"<stem>_predict.csv".
                # We cannot rename its output, so fall back to the un-namespaced name.
                wsl_video = to_wsl_mnt_path(video_win_path)
                expected_csv_win = os.path.join(output_win_dir, f"{stem}_predict.csv")

            if self.cfg.min_free_gb > 0:
                free = self._wsl_free_gb(self.cfg.wsl_stage_dir)
                if free is not None and free < self.cfg.min_free_gb:
                    logger.warning("Low WSL disk: %.1f GB free at %s (< min_free_gb=%.1f); "
                                   "consider running tools/cleanup_caches.py.",
                                   free, self.cfg.wsl_stage_dir, self.cfg.min_free_gb)

            cmd = (
                f"conda activate {self.cfg.conda_env} && "
                f"cd {self.cfg.repo_dir}/src && "
                f"python predict.py "
                f"--video_path {wsl_tilde_quote(wsl_video)} "
                f"--model_weights {wsl_tilde_quote(self.cfg.weights_path)} "
                f"--output_dir {wsl_tilde_quote(wsl_out)} "
                f"--queue_length {self.cfg.queue_length}"
            )
            logger.info("Running TrackNet inference on %s ...", base)
            try:
                res = self._wsl_bash(cmd)
            except subprocess.TimeoutExpired:
                logger.error("TrackNet inference timed out after %ss.",
self.cfg.timeout_sec)
                return None

            if res.returncode != 0:
                logger.error("TrackNet predict.py failed:\n%s", (res.stderr or
res.stdout)[-2000:])
                return None

            if not os.path.exists(expected_csv_win):
                logger.error("TrackNet finished but expected CSV not found at %s",
```

```
expected_csv_win)
                return None
            logger.info("TrackNet trajectory CSV: %s", expected_csv_win)

            # Success ? the CSV is the durable output; drop the staged video copy (a pure,
            # regenerable intermediate). On earlier failure we return above without
cleaning.
            if self.cfg.stage_in_wsl and not self.cfg.keep_frames:
                logger.info("Cache hygiene: removing staged video for %s "
                            "(set indexing.tracknet.keep_frames=true to retain).", key)
                self._wsl_rm_rf([wsl_video])
            return expected_csv_win

        def _stage_video(self, video_win_path: str, base: str) -> Optional[str]:
            """Copy the input video into the WSL filesystem (avoids slow /mnt/c reads).

            Returns the WSL path to the staged copy, or None on failure.
            """
            src_mnt = to_wsl_mnt_path(video_win_path)
            reason = self.wsl_source_unreadable_reason(src_mnt)
            if reason:
                logger.error("Cannot stage video into WSL: %s", reason)
                return None
            dest = f"{self.cfg.wsl_stage_dir}/{base}"
            cmd = f"mkdir -p {self.cfg.wsl_stage_dir} && cp {shlex.quote(src_mnt)}
{wsl_tilde_quote(dest)} && echo OK"
            try:
                res = self._wsl_bash(cmd)
            except subprocess.TimeoutExpired:
                logger.error("Staging video into WSL timed out.")
                return None
            if res.returncode != 0 or "OK" not in (res.stdout or ""):
                logger.error("Failed to stage video into WSL: %s", (res.stderr or
res.stdout).strip())
                return None
            return dest

        # ------------------------------------------------------------ CSV -> windows

        @staticmethod
        def parse_trajectory_csv(csv_win_path: str) -> List[TrajectoryPoint]:
            """Parse a TrackNet predict CSV (columns: Frame,Visibility,X,Y)."""
            points: List[TrajectoryPoint] = []
            with open(csv_win_path, newline="") as f:
                reader = csv.DictReader(f)
                for row in reader:
                    try:
                        points.append(TrajectoryPoint(
                            frame=int(row["Frame"]),
                            visible=int(row["Visibility"]) == 1,
                            x=float(row["X"]),
                            y=float(row["Y"]),
                        ))
                    except (KeyError, ValueError):
                        continue
            points.sort(key=lambda p: p.frame)
            return points

        @staticmethod
        def _active_frames(points: List[TrajectoryPoint], fps: float, frame_width: float,
                           velocity_thresh: float) -> List[Tuple[int, float]]:
            """Per-frame ``(frame_idx, velocity)`` for frames where the shuttle is in
flight.

            Computes each visible point's normalised velocity from the last *visible* point;
```

```
261             an absent shuttle breaks the trajectory. A frame qualifies when its velocity
exceeds
262             ``velocity_thresh``. ``fps``/``frame_width`` are expected to be already
defaulted by
263             the caller so identical floats feed the velocity math."""
264             active = []  # (frame_idx, velocity)
265             prev: Optional[TrajectoryPoint] = None
266             for p in points:
267                 if not p.visible:
268                     prev = None  # break the trajectory; absent shuttle = not in flight
269                     continue
270                 if prev is not None:
271                     dframes = p.frame - prev.frame
272                     if dframes > 0:
273                         dist = ((p.x - prev.x) ** 2 + (p.y - prev.y) ** 2) ** 0.5
274                         velocity = (dist / frame_width) * (fps / dframes)
275                         if velocity > velocity_thresh:
276                             active.append((p.frame, velocity))
277                 prev = p
278             return active
279
280         @staticmethod
281         def trajectory_to_action_windows(
282             points: List[TrajectoryPoint],
283             fps: float,
284             frame_width: float,
285             chunk_start: float = 0.0,
286             velocity_thresh: float = 0.02,
287             merge_gap: float = 2.0,
288             min_window_duration: float = 2.0,
289             chunk_index: Optional[int] = None,
290             max_window_duration: float = 0.0,
291             min_split_gap: float = 0.5,
292             window_hard_cap: bool = False,
293             window_inactivity_end: float = 0.0,
294             inactivity_min_density: float = 0.34,
295             inactivity_rally_thresh: float = 0.08,
296             window_blob_fallback: bool = False,
297             window_blob_factor: float = 4.0,
298         ) -> List[Dict[str, Any]]:
299             """Convert a shuttle trajectory into candidate rally windows.
300
301             A frame is "active" when the shuttle is visible AND its inter-frame
302             displacement (normalised by frame width, per second) exceeds
303             ``velocity_thresh`` ? i.e. the shuttle is in flight. Active frames within
304             ``merge_gap`` seconds of each other are grouped into one window; short
305             windows are dropped. Mirrors HybridYoloSegmenter's windowing so the dict
306             shape feeding the AI-handoff phase is identical.
307
308             ``max_window_duration`` (0 = OFF, the default ? behaviour unchanged): a guard
against
309             the *over-merge* failure where "shuttle moving" is conflated with "rally in
progress"
310             and a single window swallows several rallies + the dead time between them. When
set, any
311             window longer than this is recursively SPLIT at its largest internal inactivity
gap (the
312             longest stretch with no in-flight shuttle ? the most likely rally boundary),
provided
313             that gap is at least ``min_split_gap`` seconds. This is the bounded-windowing
fix from
314             ``docs/RALLY_QUALITY_RESEARCH.md`` §6 (W1); it never merges, only cuts, so
recall cannot
315             drop, and it is config-gated default-OFF until measured on the golden set.
316
```

```
            ``window_hard_cap`` (default OFF) makes ``max_window_duration`` a TRUE cap: when
an
            over-long window has NO internal inactivity gap to split on (a continuously-
active
            trajectory ? the shuttle never stops moving, e.g. the real-footage
``mahadevpura-1``
            blob where the gap-splitter alone left a single 174 s window), it is recursively
            hard-chopped at its temporal midpoint until every piece is ? the cap. Still only
cuts
            (never merges), so recall cannot drop; gated default-OFF until measured.

            ``window_blob_fallback`` (default OFF ? the R5 last-resort blob splitter):
unlike
            ``window_hard_cap`` (which midpoint-chops BEFORE the R4 trim, so it fires on
every gap-less
            over-cap window and over-segments normal clips), this runs AFTER the R4
inactivity trim and
            force-chops ONLY a genuine *blob* ? a group still longer than
``window_blob_factor`` ×
            ``max_window_duration`` once the gap-split and the principled rally-end trim
have both had
            their chance (e.g. mahadevpura-1's ~65 s residual ? 11× a 6 s cap; a 7 s rally
is left
            alone). The blob is midpoint-chopped down to ? the cap, those forced cuts are
logged, and
            each resulting window is flagged ``forced_cut=True`` ? surfacing the clip as one
that needs
            rally-STATE cues (net-crossings / two-sided motion), not a bigger cap. Only cuts
(recall
            can't drop). Requires ``max_window_duration`` > 0; gated default-OFF until
measured.
            """
            if fps <= 0:
                fps = 30.0
            if frame_width <= 0:
                frame_width = 1280.0

            active = TrackNetRunner._active_frames(points, fps, frame_width,
velocity_thresh)

            if not active:
                return []

            max_gap_frames = int(merge_gap * fps)

            def _finalize(group: List[Tuple[int, float]]) -> Dict[str, Any]:
                vels = [v for _, v in group]
                return {
                    "start": group[0][0] / fps + chunk_start,
                    "end": group[-1][0] / fps + chunk_start,
                    "active_frame_count": len(group),
                    "peak_velocity": max(vels),
                    "mean_velocity": sum(vels) / len(vels),
                    "track_id_count": 1,  # single shuttle; kept for window-shape parity
                    "chunk_index": chunk_index,
                }

            groups: List[List[Tuple[int, float]]] = []
            group = [active[0]]
            for af in active[1:]:
                if af[0] - group[-1][0] <= max_gap_frames:
                    group.append(af)
                else:
                    groups.append(group)
                    group = [af]
```

```
            groups.append(group)

            if max_window_duration > 0:
                groups = TrackNetRunner._split_overlong_groups(
                    groups, int(max_window_duration * fps), int(min_split_gap * fps),
                    hard_cap=window_hard_cap)

            # R4 ? rally-END inactivity trim (default OFF). Applied AFTER the cap split so
each rally
            # piece has its own post-rally drift tail removed: the velocity windowing keeps
a window
            # "active" while the shuttle drifts/gets-picked-up above threshold, over-
extending the END
            # (the measured gap vs golden). Trim back to the last frame whose trailing
window is still
            # dense with motion (rally on); a sustained low-density run = rally over. Only
cuts.
            if window_inactivity_end > 0:
                tw = int(window_inactivity_end * fps)
                groups = [TrackNetRunner._trim_inactive_tail(g, tw, inactivity_min_density,
                                                             inactivity_rally_thresh) for g
in groups]

            # R5 ? blob-fallback (default OFF). LAST resort: after the gap-split AND the R4
trim, any
            # group still longer than the cap is a gap-less, rally-end-signal-less blob;
force-chop it to
            # ? the cap at the midpoint and flag every piece, so the degenerate single-
window outcome is
            # avoided and the clip is surfaced (logged) as needing rally-STATE cues, not a
bigger cap.
            forced_flags = [False] * len(groups)
            if window_blob_fallback and max_window_duration > 0:
                groups, forced_flags = TrackNetRunner._blob_fallback_chop(
                    groups, int(max_window_duration * fps), window_blob_factor)

            windows = []
            for g, forced in zip(groups, forced_flags):
                w = _finalize(g)
                if forced:
                    w["forced_cut"] = True
                windows.append(w)
            return [w for w in windows if (w["end"] - w["start"]) >= min_window_duration]

        @staticmethod
        def _trim_inactive_tail(group: List[Tuple[int, float]], trail_frames: int,
                                min_density: float, rally_thresh: float) -> List[Tuple[int,
float]]:
            """R4: trim the post-rally drift tail. After a rally the shuttle keeps moving
*slowly*
            (picked up / walked / knock-up) above ``velocity_thresh``, so the velocity
windowing
            over-extends the END. A frame is "fast" (real rally motion) iff its velocity >
``rally_thresh``
            (higher than the active threshold). The rally is "on" at frame ``i`` while its
trailing
            ``trail_frames`` window holds >= ``min_density`` fraction of fast frames; trim
back to the
            LAST such frame ? everything after is sustained slow drift (rally over). Only
cuts (recall
            can't rise); a window whose tail never goes slow is untouched, and if no fast-
dense point
            exists at all the group is kept whole (fail-safe, no over-trim). O(n) two-
pointer; the START
            is left alone (already ~Gemini-accurate per the n=2 boundary-error finding)."""
```

```
            if trail_frames <= 0 or len(group) < 2:
                return group
            frames = [f for f, _ in group]
            fast = [1 if v > rally_thresh else 0 for _, v in group]
            lo = 0
            fast_sum = 0
            last_dense = -1
            for i in range(len(frames)):
                fast_sum += fast[i]
                while frames[lo] <= frames[i] - trail_frames:
                    fast_sum -= fast[lo]
                    lo += 1
                win_count = i - lo + 1
                if win_count > 0 and fast_sum / win_count >= min_density:
                    last_dense = i
            return group[: last_dense + 1] if last_dense >= 0 else group

        @staticmethod
        def _midpoint_split_index(group: List[Tuple[int, float]]) -> int:
            """Index of the active frame nearest the group's temporal midpoint (used to chop
a
            gap-less group into two). Shared by the W1 hard-cap fallback and the R5 blob-
fallback."""
            mid = (group[0][0] + group[-1][0]) / 2.0
            split_i = min(range(1, len(group)), key=lambda i: abs(group[i][0] - mid))
            return split_i

        @staticmethod
        def _split_overlong_groups(groups: List[List[Tuple[int, float]]], max_win_frames:
int,
                                   min_split_frames: int,
                                   hard_cap: bool = False) -> List[List[Tuple[int, float]]]:
            """Recursively split any active-frame group longer than ``max_win_frames`` at
its largest
            internal gap (? ``min_split_frames``) ? an inactivity-aware cut at the most
likely rally
            boundary. Splitting only subdivides existing groups (never merges/extends), so
it cannot
            invent or drop coverage; a group with no gap ? the floor is left intact (a
genuinely long
            rally) UNLESS ``hard_cap`` is set, in which case such a group is hard-chopped at
its
            temporal midpoint until every piece is ? ``max_win_frames`` (makes the cap a
true upper
            bound on continuously-active trajectories). Iterative worklist to bound stack
depth."""
            if max_win_frames <= 0:
                return groups
            out: List[List[Tuple[int, float]]] = []
            work = list(groups)
            while work:
                g = work.pop()
                if len(g) < 2 or (g[-1][0] - g[0][0]) <= max_win_frames:
                    out.append(g)
                    continue
                best_i, best_gap = -1, -1
                for i in range(1, len(g)):
                    gap = g[i][0] - g[i - 1][0]
                    if gap > best_gap:
                        best_gap, best_i = gap, i
                if best_i < 0 or best_gap < min_split_frames:
                    # Too long but no real dead-time gap. Default: keep as one (a genuine
long rally).
                    # hard_cap: chop at the temporal midpoint so the cap is actually
enforced.
```

```
                    if hard_cap:
                        split_i = TrackNetRunner._midpoint_split_index(g)
                        work.append(g[:split_i])
                        work.append(g[split_i:])
                    else:
                        out.append(g)
                    continue
                work.append(g[:best_i])
                work.append(g[best_i:])
            out.sort(key=lambda grp: grp[0][0])
            return out

        @staticmethod
        def _blob_fallback_chop(groups: List[List[Tuple[int, float]]], max_win_frames: int,
                                blob_factor: float = 4.0
                                ) -> Tuple[List[List[Tuple[int, float]]], List[bool]]:
            """R5 blob splitter ? the gap-less, R4-survived residual. After the W1 gap-split
AND the R4
            inactivity trim have each had their chance, a group still longer than
``blob_factor`` ×
            ``max_win_frames`` is a genuine continuously-active blob (no internal gap, no
rally-end
            signal the principled rules could find ? the real-footage ``mahadevpura-1`` ~65
s residual ?
            11× a 6 s cap). The factor is what keeps this SURGICAL: a normal long rally
(~1?2× the cap)
            is left ALONE; only the degenerate blob qualifies. As a LAST RESORT the blob is
            midpoint-chopped to ? the cap and every resulting piece is FLAGGED as a forced
cut, so the
            degenerate single-window outcome is avoided and the clip is surfaced (logged) as
needing
            rally-STATE cues ? net-crossings / two-sided motion ? not a bigger cap. Only
cuts (recall
            can't drop). Returns ``(groups, forced_flags)`` aligned 1:1; ``forced_flags[i]``
is True iff
            that piece came from a forced chop. Iterative worklist to bound stack depth."""
            if max_win_frames <= 0:
                return groups, [False] * len(groups)
            blob_frames = max_win_frames * max(1.0, blob_factor)
            out: List[List[Tuple[int, float]]] = []
            flags: List[bool] = []
            forced = 0
            for g in groups:
                if len(g) < 2 or (g[-1][0] - g[0][0]) <= blob_frames:
                    out.append(g)
                    flags.append(False)
                    continue
                pieces: List[List[Tuple[int, float]]] = []
                work = [g]
                while work:
                    h = work.pop()
                    if len(h) < 2 or (h[-1][0] - h[0][0]) <= max_win_frames:
                        pieces.append(h)
                        continue
                    split_i = TrackNetRunner._midpoint_split_index(h)
                    work.append(h[:split_i])
                    work.append(h[split_i:])
                pieces.sort(key=lambda grp: grp[0][0])
                out.extend(pieces)
                flags.extend([True] * len(pieces))
                forced += len(pieces) - 1
            if forced:
                logger.warning(
                    "R5 blob-fallback: forced %d midpoint cut(s) on continuously-active
group(s) with "
```

```
521                     "no rally-end signal ? this clip needs rally-STATE cues (net-crossings /
two-sided "
522                     "motion), not a bigger cap", forced)
523             return out, flags
524
```

9. user (2026-06-20T08:57:21.870Z)

```
1       """NativeWasbRunner coverage ? the Linux-native (no-WSL) WASB detector path (M3.5).
2
3       The native subprocess is mocked (no torch/conda/GPU here). These tests lock the contract
4       the GPU-box byte-parity gate then relies on: it IS a DetectorRunner, it is NOT a WSL
runner
5       (no WslCommandMixin / wsl invocation), it builds a native argv (no `wsl`, no `conda
6       activate`, no `/mnt` translation) that runs the SAME wasb_infer.py with the SAME output
CSV
7       name as the WSL runner, threads the device through, and cleans the frame cache on
success.
8       """
9       import os
10      import types
11      from unittest.mock import patch
12
13      import pytest
14
15      from backend.pipeline.detectors.native_wasb_runner import NativeWasbRunner,
NativeWasbConfig
16      from backend.pipeline.detectors.base import DetectorRunner, WslCommandMixin
17
18
19      def _proc(rc=0, stdout="", stderr=""):
20          return types.SimpleNamespace(returncode=rc, stdout=stdout, stderr=stderr)
21
22
23      # --- identity / no-WSL guarantee
------------------------------------------------------
24      def test_is_a_detector_runner():
25          assert isinstance(NativeWasbRunner(NativeWasbConfig()), DetectorRunner)
26
27
28      def test_is_not_a_wsl_runner():
29          """The whole point of M3.5: the native runner must carry NO WSL plumbing."""
30          r = NativeWasbRunner(NativeWasbConfig())
31          assert not isinstance(r, WslCommandMixin)
32          assert not hasattr(r, "_wsl_bash")
33
34
35      def test_reuses_model_agnostic_windowing():
36          from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
37          assert NativeWasbRunner.parse_trajectory_csv is TrackNetRunner.parse_trajectory_csv
38          assert NativeWasbRunner.trajectory_to_action_windows is
TrackNetRunner.trajectory_to_action_windows
39
40
41      # --- config resolution (env overrides win over config over default)
----------------------
42      def test_from_indexing_cfg_defaults():
43          cfg = NativeWasbConfig.from_indexing_cfg({})
44          assert cfg.device == "cuda" and cfg.python_bin == "python"
45          assert cfg.weights_path == "" and cfg.repo_dir == "~/models/WASB-SBDT"
46          assert cfg.stream_video is False and cfg.keep_frames is False
47
48
49      def test_from_indexing_cfg_reads_config_block():
50          cfg = NativeWasbConfig.from_indexing_cfg({"wasb": {
```

```
            "weights_path": "/m/w.pth.tar", "device": "cpu", "python_bin":
"/envs/wasb/bin/python",
            "repo_dir": "/m/WASB-SBDT", "stream_video": True, "keep_frames": True, "sport":
"tennis"}})
        assert cfg.weights_path == "/m/w.pth.tar" and cfg.device == "cpu"
        assert cfg.python_bin == "/envs/wasb/bin/python" and cfg.repo_dir == "/m/WASB-SBDT"
        assert cfg.stream_video is True and cfg.keep_frames is True and cfg.sport ==
"tennis"


    def test_env_overrides_beat_config(monkeypatch):
        monkeypatch.setenv("WASB_WEIGHTS", "/env/w.pth.tar")
        monkeypatch.setenv("WASB_DEVICE", "mps")
        monkeypatch.setenv("WASB_PYTHON", "/env/py")
        monkeypatch.setenv("WASB_REPO_DIR", "/env/repo")
        monkeypatch.setenv("WASB_SCRATCH", "/env/scratch")
        cfg = NativeWasbConfig.from_indexing_cfg({"wasb": {
            "weights_path": "/cfg/w", "device": "cpu", "python_bin": "/cfg/py", "repo_dir":
"/cfg/repo"}})
        assert cfg.weights_path == "/env/w.pth.tar" and cfg.device == "mps"
        assert cfg.python_bin == "/env/py" and cfg.repo_dir == "/env/repo"
        assert cfg.scratch_dir == "/env/scratch"


    # --- healthcheck
------------------------------------------------------------------------
    def _ok_cfg(tmp_path, **kw):
        w = tmp_path / "w.pth.tar"
        w.write_text("weights")
        (tmp_path / "repo" / "src").mkdir(parents=True)
        return NativeWasbConfig(weights_path=str(w), repo_dir=str(tmp_path / "repo"), **kw)


    def test_healthcheck_ok(tmp_path):
        r = NativeWasbRunner(_ok_cfg(tmp_path, device="cuda"))
        with patch.object(r, "_run", return_value=_proc(0, stdout="torch 1.11.0 cuda
True")):
            ok, msg = r.healthcheck()
        assert ok and "torch" in msg


    def test_healthcheck_empty_weights():
        ok, msg = NativeWasbRunner(NativeWasbConfig(weights_path="")).healthcheck()
        assert not ok and "weights" in msg.lower()


    def test_healthcheck_weights_missing(tmp_path):
        (tmp_path / "repo" / "src").mkdir(parents=True)
        cfg = NativeWasbConfig(weights_path=str(tmp_path / "nope.pth"),
repo_dir=str(tmp_path / "repo"))
        ok, msg = NativeWasbRunner(cfg).healthcheck()
        assert not ok and "not found" in msg.lower()


    def test_healthcheck_repo_missing(tmp_path):
        w = tmp_path / "w.pth.tar"
        w.write_text("x")
        cfg = NativeWasbConfig(weights_path=str(w), repo_dir=str(tmp_path / "absent"))
        ok, msg = NativeWasbRunner(cfg).healthcheck()
        assert not ok and "repo" in msg.lower()


    def test_healthcheck_torch_import_fails(tmp_path):
        r = NativeWasbRunner(_ok_cfg(tmp_path))
        with patch.object(r, "_run", return_value=_proc(1, stderr="ModuleNotFoundError:
```

```
torch")):
109             ok, msg = r.healthcheck()
110         assert not ok and "failed" in msg.lower()
111
112
113     def test_healthcheck_cuda_unavailable_when_device_cuda(tmp_path):
114         r = NativeWasbRunner(_ok_cfg(tmp_path, device="cuda"))
115         with patch.object(r, "_run", return_value=_proc(0, stdout="torch 1.11.0 cuda
False")):
116             ok, msg = r.healthcheck()
117         assert not ok and "cuda" in msg.lower()
118
119
120     def test_healthcheck_cpu_device_ok_without_cuda(tmp_path):
121         r = NativeWasbRunner(_ok_cfg(tmp_path, device="cpu"))
122         with patch.object(r, "_run", return_value=_proc(0, stdout="torch 1.11.0 cuda
False")):
123             ok, msg = r.healthcheck()
124         assert ok  # cpu run does not require a GPU
125
126
127     def test_healthcheck_python_not_found(tmp_path):
128         r = NativeWasbRunner(_ok_cfg(tmp_path, python_bin="/no/such/python"))
129         with patch.object(r, "_run", side_effect=FileNotFoundError()):
130             ok, msg = r.healthcheck()
131         assert not ok and "python" in msg.lower()
132
133
134     # --- run_predict
---------------------------------------------------------------------
135     def _drive_success(cfg, tmp_path, vid="abc123abc123", **patches):
136         """Drive run_predict to success with the subprocess mocked; returns (runner, out,
rm_spy, argv, cwd)."""
137         r = NativeWasbRunner(cfg)
138         key = f"clip__{vid[:12]}"
139         (tmp_path / f"{key}_wasb.csv").write_text("Frame,Visibility,X,Y\n")  # success = CSV
exists
140         with patch.object(r, "_copy_infer_script", return_value=True), \
141              patch.object(r, "_run", return_value=_proc(0)) as run_spy, \
142              patch.object(r, "_rm_rf") as rm_spy:
143             out = r.run_predict(r"C:\v\clip.mp4", str(tmp_path), video_id=vid)
144         argv = run_spy.call_args[0][0]
145         cwd = run_spy.call_args.kwargs.get("cwd")
146         return r, out, rm_spy, argv, cwd, key
147
148
149     def test_run_predict_builds_native_argv_no_wsl(tmp_path):
150         _, out, rm, argv, cwd, key =
_drive_success(NativeWasbConfig(weights_path="/m/w.pth", device="cuda"), tmp_path)
151         assert out == os.path.join(os.path.abspath(str(tmp_path)), f"{key}_wasb.csv")
152         # No WSL / conda plumbing anywhere in the command.
153         assert "wsl" not in argv
154         assert not any("conda" in a or "/mnt/" in a for a in argv)
155         # It runs the SAME inference core, device-threaded, writing the SAME CSV name as
WSL.
156         assert argv[0] == "python" and argv[1] == "wasb_infer.py"
157         assert "--device" in argv and argv[argv.index("--device") + 1] == "cuda"
158         assert "--weights" in argv and argv[argv.index("--weights") + 1] == "/m/w.pth"
159         assert argv[argv.index("--out") + 1] == out
160         # Disk mode extracts frames + cleans them on success.
161         assert "--frames_out_dir" in argv and "--stream-video" not in argv
162         assert cwd.endswith(os.path.join("src"))
163         rm.assert_called_once()
164
165
```

```
    def test_run_predict_stream_mode_no_frames_dir(tmp_path):
        _, out, rm, argv, _cwd, _key = _drive_success(
            NativeWasbConfig(weights_path="/m/w.pth", stream_video=True), tmp_path)
        assert out is not None
        assert "--stream-video" in argv and "--frames_out_dir" not in argv
        rm.assert_not_called()  # streaming never materializes a frame cache


    def test_run_predict_device_threaded_cpu(tmp_path):
        _, _out, _rm, argv, _cwd, _key = _drive_success(
            NativeWasbConfig(weights_path="/m/w.pth", device="cpu"), tmp_path)
        assert argv[argv.index("--device") + 1] == "cpu"


    def test_run_predict_keep_frames_skips_cleanup(tmp_path):
        _, _out, rm, _argv, _cwd, _key = _drive_success(
            NativeWasbConfig(weights_path="/m/w.pth", keep_frames=True), tmp_path)
        rm.assert_not_called()


    def test_run_predict_none_when_copy_infer_fails(tmp_path):
        r = NativeWasbRunner(NativeWasbConfig(weights_path="/m/w.pth"))
        with patch.object(r, "_copy_infer_script", return_value=False):
            assert r.run_predict(r"C:\v\clip.mp4", str(tmp_path), video_id="abc123abc123")
is None


    def test_run_predict_none_when_subprocess_fails(tmp_path):
        r = NativeWasbRunner(NativeWasbConfig(weights_path="/m/w.pth"))
        with patch.object(r, "_copy_infer_script", return_value=True), \
             patch.object(r, "_run", return_value=_proc(1, stderr="CUDA error")):
            assert r.run_predict(r"C:\v\clip.mp4", str(tmp_path), video_id="abc123abc123")
is None


    def test_run_predict_none_when_csv_absent(tmp_path):
        # Subprocess "succeeds" (rc 0) but writes no CSV -> treated as failure.
        r = NativeWasbRunner(NativeWasbConfig(weights_path="/m/w.pth"))
        with patch.object(r, "_copy_infer_script", return_value=True), \
             patch.object(r, "_run", return_value=_proc(0)):
            assert r.run_predict(r"C:\v\clip.mp4", str(tmp_path), video_id="abc123abc123")
is None


    def test_run_predict_times_out_returns_none(tmp_path):
        import subprocess
        r = NativeWasbRunner(NativeWasbConfig(weights_path="/m/w.pth"))
        with patch.object(r, "_copy_infer_script", return_value=True), \
             patch.object(r, "_run", side_effect=subprocess.TimeoutExpired(cmd="x",
timeout=1)):
            assert r.run_predict(r"C:\v\clip.mp4", str(tmp_path), video_id="abc123abc123")
is None


    def test_csv_name_matches_wsl_contract(tmp_path):
        """The native CSV name must equal the WSL runner's so downstream + the parity diff
line up."""
        _, out, _rm, _argv, _cwd, key =
_drive_success(NativeWasbConfig(weights_path="/m/w.pth"), tmp_path)
        assert os.path.basename(out) == f"{key}_wasb.csv"


    # --- the detector_impl='native' seam (parallels the stub seam)
-------------------------
    def test_resolve_runner_native_selects_native_runner_for_wasb(monkeypatch):
```

```
223         monkeypatch.delenv("RALLY_DETECTOR_IMPL", raising=False)
224         from backend.pipeline.segmenters.wasb_hybrid import WasbHybridSegmenter
225         runner, params = WasbHybridSegmenter(None, {})._resolve_runner({"detector_impl":
"native"})
226         assert isinstance(runner, NativeWasbRunner)
227         assert params.get("detector_impl") == "native"
228
229
230     def test_resolve_runner_native_env_overrides_config(monkeypatch):
231         monkeypatch.setenv("RALLY_DETECTOR_IMPL", "native")
232         from backend.pipeline.segmenters.wasb_hybrid import WasbHybridSegmenter
233         runner, _ = WasbHybridSegmenter(None, {})._resolve_runner({"detector_impl": "auto"})
234         assert isinstance(runner, NativeWasbRunner)
235
236
237     def test_resolve_runner_native_supported_for_fusion_wasb(monkeypatch):
238         monkeypatch.delenv("RALLY_DETECTOR_IMPL", raising=False)
239         from backend.pipeline.segmenters.fusion_hybrid import FusionSegmenter
240         runner, params = FusionSegmenter(None, {})._resolve_runner({"detector_impl":
"native"})
241         assert isinstance(runner, NativeWasbRunner)
242         assert params.get("fusion_substrate") == "wasb"
243
244
245     def test_resolve_runner_native_refused_for_fusion_tracknet(monkeypatch):
246         monkeypatch.delenv("RALLY_DETECTOR_IMPL", raising=False)
247         from backend.pipeline.segmenters.fusion_hybrid import FusionSegmenter
248         with pytest.raises(NotImplementedError):
249             FusionSegmenter(None, {})._resolve_runner(
250                 {"detector_impl": "native", "fusion": {"substrate": "tracknet"}})
251
252
253     def test_resolve_runner_native_refused_for_tracknet(monkeypatch):
254         monkeypatch.delenv("RALLY_DETECTOR_IMPL", raising=False)
255         from backend.pipeline.segmenters.tracknet_hybrid import TrackNetHybridSegmenter
256         with pytest.raises(NotImplementedError):
257             TrackNetHybridSegmenter(None, {})._resolve_runner({"detector_impl": "native"})
258
```

10. assistant (2026-06-20T08:57:25.004Z)

Now I have the full picture for the core parity comparison. Let me focus on the key divergence points. First, let me check the WSL runner's argv construction in detail versus the native one, and check the git diff to understand what changed in wasb_infer.py.

11. user (2026-06-20T08:57:26.701Z)

```
diff --git a/backend/pipeline/detectors/wasb_infer.py b/backend/pipeline/detectors/wasb_infer.py
index 7b589d5..5df5939 100644
--- a/backend/pipeline/detectors/wasb_infer.py
+++ b/backend/pipeline/detectors/wasb_infer.py
@@ -53,16 +53,22 @@ _DET_FILE = "det_raw.jsonl"
 _MANIFEST_FILE = "manifest.json"


-def _build_cfg(weights: str, sport: str):
-    """Compose the WASB eval config via Hydra, overriding model/weights/device."""
+def _build_cfg(weights: str, sport: str, device: str = "cuda"):
+    """Compose the WASB eval config via Hydra, overriding model/weights/device.
+
+    ``device`` is ``cuda`` (default ? the historical WSL behaviour, byte-parity preserved),
+    ``mps`` (Apple) or ``cpu``. Only the single-GPU CUDA path requests ``runner.gpus=[0]``;
+    cpu/mps run with no GPU list so the detector places tensors on ``device``.
+    """
     from hydra import compose, initialize
```

```
+    gpus = "[0]" if device == "cuda" else "[]"   # single GPU on cuda; none for cpu/mps
     with initialize(config_path="configs", version_base=None):
         cfg = compose(config_name="eval", overrides=[
             f"dataset={sport}",
             "model=wasb",
             f"detector.model_path={weights}",
-            "runner.device=cuda",
-            "runner.gpus=[0]",      # this box has a single GPU; default config assumes more
+            f"runner.device={device}",
+            f"runner.gpus={gpus}",  # default config assumes multiple GPUs; pin to this box
         ])
     return cfg

@@ -129,8 +135,13 @@ def read_manifest(cache_dir: str):


 def manifest_compatible(manifest: dict, *, frames_dir: str, weights: str, sport: str,
-                        frames_in: int, frames_total: int) -> bool:
-    """A cache is reusable only if the run that produced it matches this run's inputs."""
+                        frames_in: int, frames_total: int, device: str = "cuda") -> bool:
+    """A cache is reusable only if the run that produced it matches this run's inputs.
+
+    ``device`` defaults to ``cuda`` so caches written before device-keying (no ``device``
+    field) stay compatible with a CUDA run ? but a cpu/mps run will NOT reuse a cuda cache
+    (detections can differ numerically across devices), and vice-versa.
+    """
     if not manifest or manifest.get("version") != CACHE_VERSION:
         return False
     return (
@@ -139,6 +150,7 @@ def manifest_compatible(manifest: dict, *, frames_dir: str, weights: str,
sport:
         and manifest.get("sport") == sport
         and manifest.get("frames_in") == frames_in
         and manifest.get("frames_total") == frames_total
+        and manifest.get("device", "cuda") == device
     )


@@ -360,7 +372,8 @@ def run(frames, weights: str, out_csv: str, sport: str = "badminton",
         limit: int = 0, batch_size: int = 8, num_workers: int = 4,
         log_every_batches: int = 50, cache_dir: "str | None" = None,
         flush_every: int = 1000, fresh: bool = False,
-        frame_loader=None, source_key: "str | None" = None) -> str:
+        frame_loader=None, source_key: "str | None" = None,
+        device: str = "cuda") -> str:
     """Run WASB detection+tracking over an ordered frame source -> trajectory CSV.

     Two equivalent ways to supply pixels (output is bit-identical between them):
@@ -391,7 +404,7 @@ def run(frames, weights: str, out_csv: str, sport: str = "badminton",

     streaming = frame_loader is not None

-    cfg = _build_cfg(weights, sport)
+    cfg = _build_cfg(weights, sport, device)
     frames_in = int(cfg["model"]["frames_in"])
     input_wh = (int(cfg["model"]["inp_width"]), int(cfg["model"]["inp_height"]))
     output_wh = (int(cfg["model"]["out_width"]), int(cfg["model"]["out_height"]))
@@ -427,7 +440,7 @@ def run(frames, weights: str, out_csv: str, sport: str = "badminton",
     manifest = None if fresh else read_manifest(cache_dir)
     resume = manifest_compatible(
         manifest or {}, frames_dir=cache_key, weights=weights, sport=sport,
-        frames_in=frames_in, frames_total=len(frames),
+        frames_in=frames_in, frames_total=len(frames), device=device,
     )
     if resume:
```

```
         windows_done, det_by_fid = load_and_clean_cache(cache_dir)
@@ -445,6 +458,7 @@ def run(frames, weights: str, out_csv: str, sport: str = "badminton",
         "sport": sport,
         "frames_in": frames_in,
         "frames_total": len(frames),
+        "device": device,
         "windows_total": windows_total,
         "windows_done": windows_done,
     }
@@ -630,7 +644,8 @@ def make_video_frame_loader(video_path: str):
 def run_video_streaming(video_path: str, weights: str, out_csv: str, sport: str = "badminton",
                         limit: int = 0, batch_size: int = 8,
                         log_every_batches: int = 50, cache_dir: "str | None" = None,
-                        flush_every: int = 1000, fresh: bool = False) -> str:
+                        flush_every: int = 1000, fresh: bool = False,
+                        device: str = "cuda") -> str:
     """Fast path: decode ``video_path`` on the fly and feed frames straight to the
     detector, WITHOUT extracting every frame to a PNG first (the disk round-trip that
     dominates a long clip). Output is bit-identical to the frames-on-disk path; see
@@ -648,7 +663,7 @@ def run_video_streaming(video_path: str, weights: str, out_csv: str, sport:
str
                   batch_size=batch_size, num_workers=0,
                   log_every_batches=log_every_batches, cache_dir=cache_dir,
                   flush_every=flush_every, fresh=fresh,
-                  frame_loader=frame_loader, source_key=source_key)
+                  frame_loader=frame_loader, source_key=source_key, device=device)
     finally:
         release()

@@ -664,6 +679,8 @@ def main():
     ap.add_argument("--weights", required=True)
     ap.add_argument("--out", required=True)
     ap.add_argument("--sport", default="badminton")
+    ap.add_argument("--device", default="cuda", choices=["cuda", "mps", "cpu"],
+                    help="torch device for the detector (default cuda = the WSL parity path)")
     ap.add_argument("--limit", type=int, default=0, help="cap #frames (for quick tests)")
     ap.add_argument("--batch-size", type=int, default=8)
     ap.add_argument("--num-workers", type=int, default=4)
@@ -681,7 +698,7 @@ def main():
         run_video_streaming(args.video, args.weights, args.out, args.sport, args.limit,
                             batch_size=args.batch_size,
                             log_every_batches=args.log_every_batches, cache_dir=args.cache_dir,
-                            flush_every=args.flush_every, fresh=args.fresh)
+                            flush_every=args.flush_every, fresh=args.fresh, device=args.device)
         return

     frames_dir = args.frames_dir
@@ -692,7 +709,7 @@ def main():
     run(frames_dir, args.weights, args.out, args.sport, args.limit,
         batch_size=args.batch_size, num_workers=args.num_workers,
         log_every_batches=args.log_every_batches, cache_dir=args.cache_dir,
-        flush_every=args.flush_every, fresh=args.fresh)
+        flush_every=args.flush_every, fresh=args.fresh, device=args.device)


 if __name__ == "__main__":
```

12. assistant (2026-06-20T08:57:33.932Z)

The wasb_infer.py diff is clean and the `device` default of `cuda` preserves the prior Hydra overrides exactly. Now the critical question: the **WSL `--frames_out_dir` default differs from native**. Let me look very carefully at the WSL argv vs native argv, particularly around the `--frames_out_dir` semantics and the manifest `frames_dir`/`source_key` keying for the disk path (the parity gate's default path).

Let me trace the `frames_out_dir` path carefully. The WSL runner passes `--frames_out_dir ~/clips/{key}_frames`. The native runner passes `--frames_out_dir {scratch}/{key}_frames`. The cache key for the disk path is `osp.abspath(frames_dir)` ? which differs between WSL (`/home/user/clips/...`) and native (`/some/native/scratch/...`). But the manifest key only affects *resume*, not the trajectory output. Let me verify that the disk-path manifest key cannot affect the CSV bytes, and examine the WSL `--frames_out_dir` default carefully.

13. user (2026-06-20T08:57:35.416Z)

```
226:    It mirrors the on-disk names ``extract_frames`` writes (``{i:08d}.png``) so the
561:def extract_frames(video_path: str, frames_dir: str) -> str:
675:    ap.add_argument("--frames_out_dir", help="where to extract frames when --video is used")
696:        if not args.video:
698:        run_video_streaming(args.video, args.weights, args.out, args.sport, args.limit,
704:    frames_dir = args.frames_dir
705:    if args.video:
706:        frames_dir = extract_frames(args.video, args.frames_out_dir or (args.video +
"_frames"))
```

14. assistant (2026-06-20T08:57:43.012Z)

Now the key parity-relevant facts for the **disk path** (the default, since `stream_video=False`):

1. WSL: `--video {staged}` + `--frames_out_dir ~/clips/{key}_frames`. Frames extracted from the **staged video copy** into WSL's home; `frames_dir` = `~/clips/{key}_frames` ? manifest `frames_dir` = abspath of that.
2. Native: `--video {abspath(orig)}` + `--frames_out_dir {scratch}/{key}_frames`. `frames_dir` = native scratch path ? manifest `frames_dir` = abspath of that.

The manifest `frames_dir` differs between the two, but that only affects the resumable cache, not the trajectory bytes (the frames produced are identical PNGs; `glob.glob` + `sorted` yields the same ordered list). So **disk-path CSV bytes are identical** ? good.

Now let me check the critical `os.path.abspath` cross-platform concern and verify the WSL `--out` vs native `--out` give the same filename. Let me also check the argv ordering difference ? WSL puts `--out` last with the dir-joined path, native joins `output_dir` with the key. Let me check whether `os.path.abspath` of the video matters (cwd is the repo src for both). Let me examine the worker and config changes, and the trajectory_hybrid wiring.

15. user (2026-06-20T08:57:43.940Z)

```
1       """Shared engine for trajectory-substrate hybrid segmenters.
2
3       tracknet_hybrid and wasb_hybrid were copy-paste twins ? CODE_MAP carried a
4       standing footgun: "any P3/P4 fix must be applied to BOTH". This base class is
5       now the single home for the whole pipeline shape:
6
7           healthcheck gate -> P2 trajectory -> candidate windows (+ persistence)
8           -> P3 AI-provider handoff over padded clips -> P4 DB population/linking
9
10      A concrete subclass supplies only its identity and runner wiring:
11      - ``family``        ? config section under ``indexing.*``, output subdir, and
12                            the ``<family>-hybrid-<model>`` detector tag in metadata.
13      - ``display_label`` ? human label for log lines ("WASB", "TrackNet").
14      - ``name`` / ``description`` properties (the registry contract).
15      - ``_build_runner(idx_cfg)`` ? returns (DetectorRunner, detector-specific
16                            detection_params extras such as weights_path).
17
18      This base is deliberately NOT registered; only concrete subclasses register.
19      """
20      import os
21      import time
22      import logging
```

```
    import concurrent.futures
    from typing import Any, Dict, List, Optional, Tuple

    import cv2

    from backend.pipeline.segmenters.base import BasePlaySegmenter
    from backend.utils.video import get_video_duration, extract_video_chunk
    from backend.sports import sport_registry
    from backend.providers import provider_registry
    from backend.pipeline.detectors.base import DetectorRunner

    logger = logging.getLogger(__name__)


    def _fmt_ts(seconds: float) -> str:
        seconds = max(0, int(seconds))
        h, rem = divmod(seconds, 3600)
        m, s = divmod(rem, 60)
        return f"{h:02d}:{m:02d}:{s:02d}"


    class TrajectoryHybridSegmenter(BasePlaySegmenter):
        """Trajectory-detector hybrid: precise candidate rallies + AI semantic
boundaries."""

        #: config section under `indexing` (velocity_threshold lives there), the
        #: output subdir for trajectories, and the metadata detector-tag prefix.
        family: str = ""
        #: human-readable label used in log lines.
        display_label: str = ""

        def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
Any]]:
            """Return (runner, detector-specific detection_params extras) for the default
(WSL) path."""
            raise NotImplementedError

        def _build_native_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner,
Dict[str, Any]]:
            """Return (runner, extras) for the Linux-native (no-WSL) path. Only families
with a
            native runner override this; the base refuses with a clear message (M3.5: WASB
only)."""
            raise NotImplementedError(
                f"detector_impl='native' is not supported for the "
                f"{self.display_label or self.family or 'trajectory'!r} family ? only WASB
has a "
                f"Linux-native runner. Use detector_impl='auto' (WSL) or 'stub'.")

        def _resolve_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner,
Dict[str, Any]]:
            """Resolve the detector runner, honouring the CPU-stub and Linux-native
overrides.

            ``RALLY_DETECTOR_IMPL`` env var (takes precedence) or
``indexing.detector_impl``:
            ``"stub"`` swaps in a deterministic CPU runner (no GPU/WSL) so the whole
pipeline
            is regression-testable without a GPU ? "mock a GPU with a CPU"; ``"native"``
swaps
            in the Linux-native WASB runner (no WSL/conda-activate; GPU box) via
            ``_build_native_runner``. Anything else (``"auto"``) delegates to
``_build_runner``
            (the WSL runner) so the default production path is unchanged
            (``docs/DECOUPLED_COMPUTE/``).
```

```
75              """
76              impl = (os.environ.get("RALLY_DETECTOR_IMPL") or idx_cfg.get("detector_impl") or
"auto").lower()
77              if impl == "stub":
78                  from backend.pipeline.detectors.stub_runner import StubDetectorRunner,
StubConfig
79                  logger.info("%s: detector_impl=stub ? CPU mock runner (no GPU/WSL).",
80                              self.display_label or "trajectory")
81                  return StubDetectorRunner(StubConfig.from_indexing_cfg(idx_cfg)),
{"detector_impl": "stub"}
82              if impl in ("native", "native_wasb", "wasb_native"):
83                  logger.info("%s: detector_impl=native ? Linux-native runner (no WSL).",
84                              self.display_label or "trajectory")
85                  return self._build_native_runner(idx_cfg)
86              return self._build_runner(idx_cfg)
87
88          @staticmethod
89          def _probe_fps_width(video_path: str) -> tuple:
90              """Return (fps, frame_width) for a video, with sensible fallbacks."""
91              cap = cv2.VideoCapture(video_path)
92              fps = cap.get(cv2.CAP_PROP_FPS) if cap.isOpened() else 0.0
93              width = cap.get(cv2.CAP_PROP_FRAME_WIDTH) if cap.isOpened() else 0.0
94              cap.release()
95              return (fps if fps > 0 else 30.0, width if width > 0 else 1280.0)
96
97          @staticmethod
98          def _shot_count_from(segment: Dict[str, Any], duration: float) -> int:
99              """Provider-reported exchange count, else a duration-based estimate.
100
101             One canonical implementation ? the twins had drifted (wasb relied on
102             ``int(None)`` raising into the fallback; same result, by accident).
103             """
104             shot_count = segment.get("shuttle_exchange_count")
105             if shot_count is None:
106                 return max(3, int(duration / 0.8))
107             try:
108                 return int(shot_count)
109             except (ValueError, TypeError):
110                 return max(3, int(duration / 0.8))
111
112         # --- Fusion seams (identity by default ? wasb_hybrid/tracknet_hybrid unchanged)
-------
113         def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
114                                     frame_width: float, video_path: str,
115                                     idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
116             """Stats dict persisted into ``candidate_segments.stats`` for one window. The
BASE
117             returns exactly the 4 motion keys, so the trajectory twins persist byte-
identical
118             rows; ``FusionSegmenter`` overrides this to add ``net_crossings`` (+ person-cue
119             features). ``points`` are the whole-video trajectory points
(TrajectoryPoint)."""
120             return {
121                 "peak_velocity": cw["peak_velocity"],
122                 "mean_velocity": cw["mean_velocity"],
123                 "active_frame_count": cw["active_frame_count"],
124                 "track_id_count": cw["track_id_count"],
125             }
126
127         def _select_for_handoff(self, windows: List[Dict[str, Any]],
128                                 window_stats: List[Dict[str, Any]],
129                                 idx_cfg: Dict[str, Any]) -> List[int]:
130             """Indices of the candidate windows that proceed to the AI handoff (and thus may
131             become play_segments). The BASE sends ALL windows (no gating ? twins unchanged);
132             ``FusionSegmenter`` overrides this to apply Gate A/B when thresholds are raised.
```

```
            Persisted candidates are NOT affected ? they remain the full superset for
offline
            re-scoring."""
            return list(range(len(windows)))

        def process_video(self, video_id: str, video_path: str,  # type: ignore[override]
                          player_pool: Optional[List[str]] = None,
                          max_frames: Optional[int] = None) -> List[Dict[str, Any]]:
            # override-ignore: the ABC declares the Result contract (Success|Failure)
            # but every live segmenter still returns a bare list ? the documented
            # deliberate-incremental gap (CODE_MAP gotchas; main.py handles both).
            label = self.display_label
            # --- Sport profile + AI provider wiring (identical across segmenters) ---
            sport_name = self.config.get("sport", "badminton")
            try:
                sport_profile = sport_registry.get(sport_name)
            except KeyError as e:
                logger.error(str(e))
                return []

            idx_cfg = self.config.get("indexing", {})
            # Fully-local, NO-AI mode (default OFF): the trajectory candidate windows become
the
            # rallies directly ? Phase 3's AI handoff is skipped, so no provider / API key
is
            # needed and nothing is uploaded. Used to run the local detector standalone
(e.g. to
            # measure it against the Gemini path, or to avoid the cloud entirely). With
            # FusionSegmenter the windows are still Gate-A/B-filtered via
`_select_for_handoff`.
            skip_ai = bool(idx_cfg.get("skip_ai_handoff", False))
            provider_name = self.config.get("ai_provider", idx_cfg.get("ai_provider",
"gemini"))
            if skip_ai:
                model_name = "local-noai"
                logger.info("%s in FULLY-LOCAL mode (skip_ai_handoff=True): candidate
windows will "
                            "be emitted as rallies; no %s handoff, no API key needed.",
                            label, provider_name)
            else:
                model_name = self.config.get("ai_model", idx_cfg.get("ai_model",
"gemini-2.5-flash"))
                api_key_env_var = f"{provider_name.upper()}_API_KEY"
                api_key = os.environ.get(api_key_env_var)
                if not api_key:
                    from backend.pipeline.segmenters._keyhint import api_key_hint
                    logger.error(
                        f"{api_key_env_var} environment variable is not set! "
                        f"To run the {provider_name} handoff, set your API key. "
                        + api_key_hint(api_key_env_var)
                    )
                    return []
                try:
                    provider = provider_registry.get(provider_name, api_key=api_key,
model_name=model_name)
                except KeyError as e:
                    logger.error(str(e))
                    return []

            video_duration = get_video_duration(video_path)
            if video_duration <= 0:
                logger.error("Invalid video duration.")
                return []
            fps, frame_width = self._probe_fps_width(video_path)

```

```
            # --- Runner config + windowing thresholds ---
            # _resolve_runner honours the CPU-stub override (RALLY_DETECTOR_IMPL /
            # indexing.detector_impl="stub") so the pipeline runs GPU/WSL-free; otherwise
            # it delegates to the subclass _build_runner (the real WSL/native runner).
            runner, runner_params = self._resolve_runner(idx_cfg)

            velocity_thresh = idx_cfg.get(self.family, {}).get("velocity_threshold", 0.02)
            merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)
            min_window_duration = idx_cfg.get("min_action_window_duration", 2.0)
            # Bounded-windowing over-merge fix (W1): 0 = OFF (unchanged). See
IndexingConfig.
            max_window_duration = idx_cfg.get("max_window_duration", 0.0)
            window_min_split_gap = idx_cfg.get("window_min_split_gap", 0.5)
            window_hard_cap = idx_cfg.get("window_hard_cap", False)
            window_inactivity_end = idx_cfg.get("window_inactivity_end", 0.0)
            inactivity_rally_thresh = idx_cfg.get("inactivity_rally_thresh", 0.08)
            inactivity_min_density = idx_cfg.get("inactivity_min_density", 0.34)
            window_blob_fallback = idx_cfg.get("window_blob_fallback", False)
            window_blob_factor = idx_cfg.get("window_blob_factor", 4.0)
            handoff_padding = idx_cfg.get("ai_handoff_padding", 2.0)
            ai_max_workers = idx_cfg.get("ai_max_workers", 5)
            log_candidates = idx_cfg.get("log_yolo_candidates", True)
            store_candidates = idx_cfg.get("store_yolo_candidates", True)

            detection_params = {
                "detector": self.name,
                "velocity_thresh": velocity_thresh,
                "merge_gap": merge_gap,
                "min_action_window_duration": min_window_duration,
                **runner_params,
            }

            # --- Phase 1: env healthcheck (fail fast with a clear message) ---
            ok, msg = runner.healthcheck()
            if not ok:
                logger.error("%s detector environment not ready: %s", label, msg)
                return []
            logger.info("%s detector env OK: %s", label, msg)

            # --- Phase 2: trajectory -> candidate rally windows ---
            # Trajectory detectors process the whole video in one pass (they carry
            # their own frame buffering), so unlike yolo_hybrid we don't pre-chunk.
            t_start = time.time()
            out_dir = os.path.join("output", self.family)
            csv_path = runner.run_predict(video_path, out_dir, video_id=video_id)
            if not csv_path:
                logger.error("%s produced no trajectory; aborting.", label)
                return []

            points = runner.parse_trajectory_csv(csv_path)
            logger.info("Parsed %d trajectory points (%d visible).",
                        len(points), sum(1 for p in points if p.visible))

            all_candidate_windows = runner.trajectory_to_action_windows(
                points, fps=fps, frame_width=frame_width, chunk_start=0.0,
                velocity_thresh=velocity_thresh, merge_gap=merge_gap,
                min_window_duration=min_window_duration, chunk_index=0,
                max_window_duration=max_window_duration, min_split_gap=window_min_split_gap,
                window_hard_cap=window_hard_cap,
                window_inactivity_end=window_inactivity_end,
                inactivity_rally_thresh=inactivity_rally_thresh,
                inactivity_min_density=inactivity_min_density,
                window_blob_fallback=window_blob_fallback,
                window_blob_factor=window_blob_factor,
            )
```

```
            logger.info("Phase 2 (%s) completed in %.1fs. Candidate windows: %d",
                        label, time.time() - t_start, len(all_candidate_windows))

            if log_candidates:
                for cw in all_candidate_windows:
                    logger.info(f"[{label} rally]
{_fmt_ts(cw['start'])}-{_fmt_ts(cw['end'])} "
                                f"({cw['end'] - cw['start']:.1f}s) |
frames={cw['active_frame_count']} "
                                f"peak_v={cw['peak_velocity']:.3f}
mean_v={cw['mean_velocity']:.3f}")

            # Per-window stats via the enrichment hook ? base = the 4 motion keys; the
fusion
            # segmenter adds net_crossings + person-cue features. Computed once, reused for
both
            # persistence and the handoff gate below.
            window_stats = [
                self._enrich_candidate_stats(cw, points, fps, frame_width, video_path,
idx_cfg)
                for cw in all_candidate_windows
            ]

            # Persist raw candidates for the fine-tuning dataset (same table as YOLO).
            candidate_ids: List[Optional[int]] = [None] * len(all_candidate_windows)
            if store_candidates:
                for i, cw in enumerate(all_candidate_windows):
                    candidate_ids[i] = self.db.add_candidate_segment(
                        video_id, detector=self.name,
                        start_time=cw["start"], end_time=cw["end"],
                        chunk_index=cw["chunk_index"], confidence=min(1.0,
cw["peak_velocity"]),
                        stats=window_stats[i], detection_params=detection_params,
                    )

            if not all_candidate_windows:
                return []

            # --- Phase 3: AI provider handoff over padded candidate clips ---
            # Which windows reach the handoff (base = all; fusion may apply Gate A/B).
Persisted
            # candidates above stay the full superset ? only the served play_segments are
gated.
            handoff_indices = self._select_for_handoff(all_candidate_windows, window_stats,
idx_cfg)

            ai_segments: List[dict] = []
            if skip_ai:
                # Fully-local: the selected candidate windows ARE the rallies ? emit them
verbatim
                # (no clip extraction, no upload). Boundaries are the local detector's; shot
count
                # falls back to the duration-based estimate (no AI exchange count).
                ai_segments = [
                    {
                        "start_time": all_candidate_windows[wi]["start"],
                        "end_time": all_candidate_windows[wi]["end"],
                        "shuttle_exchange_count": None,
                        "shot_count_confidence": "LocalNoAI",
                        "_candidate_id": candidate_ids[wi],
                    }
                    for wi in handoff_indices
                ]
                logger.info("Phase 3 SKIPPED (fully-local): emitted %d candidate windows as
rallies "
```

```
                            "(no AI handoff).", len(ai_segments))
            else:
                handoff_dir = os.path.join("output", f"chunks_{provider_name}_handoff")
                os.makedirs(handoff_dir, exist_ok=True)
                logger.info("Phase 3: %s validation on %d candidate windows...",
                            provider_name, len(handoff_indices))

                def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
                    w_start = max(0, window["start"] - handoff_padding)
                    w_end = min(video_duration, window["end"] + handoff_padding)
                    w_dur = w_end - w_start
                    w_path = os.path.join(handoff_dir, f"handoff_{video_id}_{wi}.mp4")
                    if not extract_video_chunk(video_path, w_start, w_dur, w_path,
reencode=True):
                        return []
                    prompt = sport_profile.get_prompt(chunk_duration=w_dur)
                    segments, _ = provider.analyze_video(w_path, prompt,
chunk_duration=w_dur,
                                                         label=f"Handoff {wi+1}")
                    valid = []
                    for seg in segments:
                        try:
                            seg["start_time"] = float(seg["start_time"]) + w_start
                            seg["end_time"] = float(seg["end_time"]) + w_start
                            seg["_candidate_id"] = candidate_ids[wi]
                            valid.append(seg)
                        except (ValueError, TypeError, KeyError) as e:
                            logger.warning("Dropping segment with unparseable timestamps: %s
? %s", seg, e)
                    try:
                        os.remove(w_path)
                    except OSError:
                        pass
                    return valid

                with concurrent.futures.ThreadPoolExecutor(max_workers=ai_max_workers) as
executor:
                    futures = [executor.submit(process_candidate_window, wi,
all_candidate_windows[wi])
                               for wi in handoff_indices]
                    for future in futures:
                        ai_segments.extend(future.result())
                try:
                    os.rmdir(handoff_dir)
                except OSError:
                    pass
                logger.info("Phase 3 completed. AI confirmed %d semantic play segments.",
len(ai_segments))

            # --- Phase 4: DB population + candidate linking ---
            segments_data = []
            for segment in ai_segments:
                try:
                    start = float(segment["start_time"])
                    end = float(segment["end_time"])
                    dur = end - start
                    metadata = {
                        "shot_count": self._shot_count_from(segment, dur),
                        "shot_count_confidence": segment.get("shot_count_confidence",
"Unknown"),
                        "detector": f"{self.family}-hybrid-{model_name}",
                    }
                    segment_id = self.db.add_play_segment(video_id, start, end, None, None,
metadata)
                    if segment_id:
```

```
                        candidate_id = segment.get("_candidate_id")
                        if candidate_id is not None:
                            self.db.link_candidate_to_segment(candidate_id, segment_id)
                        segments_data.append({
                            "id": segment_id, "start_time": start, "end_time": end,
                            "duration": dur, "metadata": metadata,
                        })
                except Exception as e:
                    logger.warning("Skipping invalid segment struct: %s", e)

            return segments_data

```

16. user (2026-06-20T08:57:44.210Z)

```
"""WASB-based hybrid play segmenter.

Phase 2 uses **WASB** (nttcom/WASB-SBDT, MIT) shuttle-trajectory detection via
WSL2 (backend/pipeline/detectors/wasb_runner.py). WASB ships pretrained
badminton weights and runs today, so this is the live shuttle-motion substrate.

For the per-video *calibration* goal we mainly use Phase 2 (trajectory ->
candidate windows); the windowing thresholds (velocity_thresh / merge_gap /
min_window_duration) are what get tuned against golden rally labels.

The pipeline engine (healthcheck -> trajectory -> windows -> AI handoff -> DB)
lives in trajectory_hybrid.TrajectoryHybridSegmenter; this class only wires the
WASB runner and identity.
"""
from typing import Any, Dict, Tuple

from backend.pipeline.segmenters.base import segmenter_registry
from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
from backend.pipeline.detectors.base import DetectorRunner
from backend.pipeline.detectors.wasb_runner import WasbRunner, WasbConfig


class WasbHybridSegmenter(TrajectoryHybridSegmenter):
    family = "wasb"
    display_label = "WASB"

    @property
    def name(self) -> str:
        return "wasb_hybrid"

    @property
    def description(self) -> str:
        return ("Two-stage pipeline: WASB shuttle-trajectory detection (WSL2, pretrained
badminton "
                "weights) for precise candidate rallies + AI provider for semantic
boundaries.")

    def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
Any]]:
        cfg = WasbConfig.from_indexing_cfg(idx_cfg)
        return WasbRunner(cfg), {"weights_path": cfg.weights_path}

    def _build_native_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner,
Dict[str, Any]]:
        # Lazy import: the native runner pulls no heavy deps, but keep parity with the
stub
        # seam (imported only when detector_impl='native' is actually selected).
        from backend.pipeline.detectors.native_wasb_runner import NativeWasbRunner,
NativeWasbConfig
        cfg = NativeWasbConfig.from_indexing_cfg(idx_cfg)
```

```
45              return NativeWasbRunner(cfg), {"weights_path": cfg.weights_path,
46                                             "detector_impl": "native", "device": cfg.device}
47
48
49      segmenter_registry.register(WasbHybridSegmenter)
50
```

17. assistant (2026-06-20T08:57:57.192Z)

Now I have a critical finding. Let me trace the **weights override** difference, which is the most likely byte-parity divergence on `device=cuda`.

In the WSL runner (`wasb_runner.py` line 166): `--weights {wsl_tilde_quote(self.cfg.weights_path)}` where the default `weights_path = "~/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar"` (a **tilde-relative** path). `wsl_tilde_quote` keeps the `~/` so bash expands it.

In the native runner (line 161): `weights = os.path.expanduser(self.cfg.weights_path)` then passes the **absolute expanded** path. The `--weights` value goes into the Hydra override `detector.model_path={weights}`.

This is the critical parity issue: **the weights string is baked into the Hydra `detector.model_path` override AND into the manifest `weights` field**. Let me verify whether the manifest comparison would differ ? but more importantly, whether `detector.model_path` resolution differs. In WSL, the cfg gets `detector.model_path=~/models/...` (literal tilde, resolved by torch/os at load time presumably). In native, it gets `detector.model_path=/home/user/models/...` (already expanded). The *loaded weights* are the same file ? same model ? same detections. So byte-parity of the trajectory holds **as long as both paths point to the same file**. That's fine for the model. But let me check whether the manifest `weights` field being different breaks **resume** parity ? it does (cache won't cross WSL?native), but that's intended and doesn't affect a fresh-run CSV.

Now let me check the most subtle issue: the **cwd**. Let me verify both run from the repo `src/` so Hydra's `config_path="configs"` (relative) resolves identically. WSL: `cd {repo_dir}/src`. Native: `cwd=self._repo_src()` = `expanduser(repo_dir)/src`. Both resolve to the same config dir ? same Hydra config ? parity. Good.

Now let me check the worker changes and config model changes for parity-relevant issues, and check the existing cache tests to verify the device param addition doesn't break them.

18. user (2026-06-20T08:57:57.789Z)

```
1       """Worker dispatcher ? pull from storage ? run the pipeline ? push outputs back.
2
3           python -m backend.worker infer --video-uri <uri> --out-uri <prefix> \
4               [--segmenter wasb_hybrid] [--config config.json] [--set k=v ...] [--scratch DIR]
5           python -m backend.worker train ...        # Gen-0 train-from-bucket (follow-up)
6
7       The URI scheme selects the storage backend via ``backend.storage`` (a bare path /
``local://``
8       is an identity passthrough; ``s3://`` / ``gcs://`` is an object store), so existing
local runs
9       are byte-for-byte unchanged. Combine with ``RALLY_DETECTOR_IMPL=stub`` and
10      ``--set indexing.skip_ai_handoff=true`` for a **secret-free, GPU-free** run (no Gemini
key) ?
11      the path validated end-to-end on DigitalOcean (``docs/SETUP_NEW_MACHINE.md``).
12
13      This is a thin orchestrator: it composes the SAME building blocks the ``backend.main``
CLI uses
14      (validator registry ? segmenter ? report), never a forked pipeline.
15      """
16      from __future__ import annotations
17
18      import argparse
19      import csv
```

```
    import json
    import os
    import sys
    import tempfile
    from typing import Any, List

    from backend import storage
    from backend.config import load_config
    from backend.infrastructure.database import Database
    from backend.pipeline.segmenters.base import segmenter_registry
    from backend.pipeline.validators.base import registry as validator_registry
    from backend.reporting import emit_indexer_report
    from backend.results import Failure
    from backend.utils.hashing import compute_video_id


    def _coerce(v: str) -> Any:
        """Coerce a ``--set`` string value to bool/int/float where it round-trips, else
str."""
        low = v.lower()
        if low in ("true", "false"):
            return low == "true"
        for cast in (int, float):
            try:
                return cast(v)
            except ValueError:
                pass
        return v


    def _apply_overrides(config: Any, sets: List[str]) -> None:
        """Apply ``a.b.c=value`` overrides onto the typed config (e.g.
indexing.skip_ai_handoff=true)."""
        for kv in sets or []:
            key, sep, val = kv.partition("=")
            if not sep:
                raise ValueError(f"--set expects k=v, got {kv!r}")
            parts = key.split(".")
            obj = config
            for p in parts[:-1]:
                obj = getattr(obj, p)
            setattr(obj, parts[-1], _coerce(val))


    def _write_intervals_csv(segments: List[dict], path: str) -> None:
        """Decimal-second ``[start,end)`` rallies + shot count / ending reason ? the join-
friendly
        artifact (same schema as the rally-annotator labels)."""
        with open(path, "w", newline="") as f:
            w = csv.writer(f)
            w.writerow(["start", "end", "shot_count", "ending_reason"])
            for s in segments:
                w.writerow([
                    f'{float(s.get("start_time", 0.0)):.3f}',
                    f'{float(s.get("end_time", 0.0)):.3f}',
                    s.get("shot_count", ""),
                    s.get("ending_reason", s.get("shot_count_confidence", "")),
                ])


    def _write_report_json(video_id: str, segments: List[dict], status: str, path: str) ->
None:
        with open(path, "w") as f:
            json.dump({
                "video_id": video_id,
```

```
                "status": status,
                "segment_count": len(segments),
                "segments": [{"start": float(s.get("start_time", 0.0)),
                              "end": float(s.get("end_time", 0.0))} for s in segments],
            }, f, indent=2)


    def cmd_infer(args: argparse.Namespace) -> int:
        config = load_config(args.config) if args.config else load_config()
        _apply_overrides(config, args.set)
        storage_cfg = config.storage.model_dump()

        scratch = args.scratch or tempfile.mkdtemp(prefix="rally-worker-")
        os.makedirs(scratch, exist_ok=True)
        config.output.output_dir = scratch

        # 1. PULL ? local:// / bare path = identity passthrough; s3:// / gcs:// = download
to scratch.
        local_video = storage.fetch(args.video_uri, os.path.join(scratch, "in.mp4"),
config=storage_cfg)
        print(f"[worker] pulled {args.video_uri} -> {local_video}")

        # 2. IDENTITY ? whole-file MD5 (the labels?video join key; pulled bytes must be
byte-identical).
        video_id = compute_video_id(local_video)

        # 3. VALIDATE (same registry as the main CLI).
        val_results, val_context = validator_registry.run_all_with_context(local_video,
config)
        failures = [r for r in val_results if not r.passed]
        db = Database(os.path.join(scratch, config.output.db_name))
        db.add_video(video_id, local_video, "failed" if failures else "processed",
                     [r.model_dump() for r in val_results])

        segments: List[dict] = []
        segmenter_actual = None
        segmentation_ran = False
        status = "failed"
        try:
            if failures:
                print("[worker] validation FAILED: "
                      + "; ".join(f"{r.validator_name}: {r.message}" for r in failures))
                return 2
            seg_name = args.segmenter or config.indexing.default_segmenter
            segmenter_cls = segmenter_registry.get(seg_name)
            if not segmenter_cls:
                print(f"[worker] unknown segmenter: {seg_name!r} "
                      f"(have {list(segmenter_registry.list_available())})")
                return 2
            segmenter_actual = seg_name
            result = segmenter_cls(db, config).process_video(video_id, local_video,
max_frames=args.max_frames)
            segmentation_ran = True
            if isinstance(result, Failure):
                print(f"[worker] segmenter FAILED: {result.message}")
                return 3
            segments = result.value if hasattr(result, "value") else result
            status = "success"
        finally:
            # Best-effort per-video observability report (never raises) ? parity with the
main CLI.
            emit_indexer_report(
                db=db, video_id=video_id, video_path=local_video, config=config,
                val_results=val_results, val_context=val_context,
                segmenter_requested=args.segmenter, segmenter_actual=segmenter_actual,
```

```
                was_reingest=False, segmentation_ran=segmentation_ran,
            )

        # 5. SERIALIZE + 6. PUSH (outputs/<video_id>/?). idempotent on video_id.
        out_local = os.path.join(scratch, "outputs", video_id)
        os.makedirs(out_local, exist_ok=True)
        _write_intervals_csv(segments, os.path.join(out_local, "intervals.csv"))
        _write_report_json(video_id, segments, status, os.path.join(out_local,
"report.json"))

        out_prefix = args.out_uri.rstrip("/")
        pushed = []
        for fn in sorted(os.listdir(out_local)):
            uri = f"{out_prefix}/outputs/{video_id}/{fn}"
            storage.put(os.path.join(out_local, fn), uri, config=storage_cfg)
            pushed.append(uri)

        print(f"[worker] infer done: {len(segments)} segment(s); pushed {len(pushed)}
artifact(s):")
        for u in pushed:
            print("   ", u)
        return 0


    def _resolve_train_detector(args: argparse.Namespace, config: Any):
        """Build the trajectory DetectorRunner for `--trajectory-source detector`, honouring
        detector_impl (RALLY_DETECTOR_IMPL / indexing.detector_impl: auto=WSL, native=GPU
box,
        stub=CPU mock). Returns the runner, or prints an error + returns None.

        Reuses the segmenter's `_resolve_runner` seam so the worker and the live CLI build
the
        detector identically (one source of truth). A non-trajectory segmenter (e.g. yolo)
has
        no shuttle detector and is rejected.
        """
        from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter

        seg_cls = segmenter_registry.get(args.segmenter)
        if not seg_cls:
            print(f"[worker] unknown segmenter: {args.segmenter!r} "
                  f"(have {list(segmenter_registry.list_available())})")
            return None
        seg = seg_cls(None, config)
        if not isinstance(seg, TrajectoryHybridSegmenter):
            print(f"[worker] --segmenter {args.segmenter!r} has no shuttle detector; "
                  f"use a trajectory hybrid (wasb_hybrid / tracknet_hybrid /
fusion_hybrid).")
            return None
        runner, _ = seg._resolve_runner(config.get("indexing", {}))
        ok, msg = runner.healthcheck()
        if not ok:
            print(f"[worker] detector environment not ready: {msg}")
            return None
        print(f"[worker] detector ready ({args.segmenter}): {msg}")
        return runner


    def cmd_train(args: argparse.Namespace) -> int:
        """Gen-0 train-from-bucket: pull labels (+ videos) -> trajectories -> manifest ->
shell out
        to training.gen0.harness (backend must NOT import training) -> push the artifact
bundle.

        Two trajectory sources (the train pipeline + harness are identical either way):
```

```
197           * ``synth`` (default) ? the CPU "mock GPU": deterministic, label-consistent
synthetic
198             shuttle paths (no GPU/WSL), so the whole pipeline is validated on a CPU box /
CI.
199           * ``detector`` ? REAL shuttle trajectories: pull each video and run the resolved
200             DetectorRunner (native WASB on a GPU box, or stub for a GPU-free wiring test).
This
201             is the M3.5 "first real training" path; only the trajectory source flips.
202         """
203         import subprocess
204 
205         config = load_config(args.config) if args.config else load_config()
206         _apply_overrides(config, args.set)
207         storage_cfg = config.storage.model_dump()
208 
209         scratch = args.scratch or tempfile.mkdtemp(prefix="rally-train-")
210         labels_dir = os.path.join(scratch, "labels")
211         traj_dir = os.path.join(scratch, "trajectories")
212         videos_dir = os.path.join(scratch, "videos")
213         os.makedirs(labels_dir, exist_ok=True)
214         os.makedirs(traj_dir, exist_ok=True)
215 
216         corpus = args.corpus_uri.rstrip("/")
217         label_uris = sorted(u for u in storage.list_uris(f"{corpus}/labels/",
config=storage_cfg)
218                             if u.lower().endswith(".csv"))
219         if len(label_uris) < 2:
220             print(f"[worker] need >=2 labeled videos for leave-one-video-out; found
{len(label_uris)} "
221                   f"under {corpus}/labels/")
222             return 2
223 
224         # Real-detector source: resolve the runner once + index the bucket's videos/ by
stem.
225         runner = None
226         video_by_stem: dict = {}
227         if args.trajectory_source == "detector":
228             os.makedirs(videos_dir, exist_ok=True)
229             runner = _resolve_train_detector(args, config)
230             if runner is None:
231                 return 2
232             for vuri in storage.list_uris(f"{corpus}/videos/", config=storage_cfg):
233                 vname = storage.parse_uri(vuri).name
234                 video_by_stem[os.path.splitext(vname)[0]] = vuri
235 
236         print(f"[worker] trajectory source: {args.trajectory_source}")
237         specs: List[dict] = []
238         for uri in label_uris:
239             fname = storage.parse_uri(uri).name                    # e.g.
GX020094_proxy.rallies.csv
240             name = fname.replace(".rallies.csv", "").replace(".csv", "")
241             local_label = storage.fetch(uri, os.path.join(labels_dir, fname),
config=storage_cfg)
242             if args.trajectory_source == "detector":
243                 vuri = video_by_stem.get(name)
244                 if not vuri:
245                     print(f"[worker] no video for {name!r} under {corpus}/videos/ ?
skipping.")
246                     continue
247                 local_video = storage.fetch(vuri, os.path.join(videos_dir,
storage.parse_uri(vuri).name),
248                                             config=storage_cfg)
249                 video_id = compute_video_id(local_video)
250                 # Real fps/width drive the trajectory frame->time mapping (synth fixes them
at 30/1280).
```

```
                from backend.pipeline.segmenters.trajectory_hybrid import
TrajectoryHybridSegmenter
                fps, frame_width = TrajectoryHybridSegmenter._probe_fps_width(local_video)
                traj = runner.run_predict(local_video, traj_dir, video_id=video_id)
                if not traj:
                    print(f"[worker] detector produced no trajectory for {name!r} ?
skipping.")
                    continue
                specs.append({"name": name, "trajectory": traj, "labels": local_label,
                              "fps": fps, "frame_width": frame_width})
            else:
                from backend.pipeline.detectors.stub_runner import
synth_trajectory_from_labels
                traj = os.path.join(traj_dir, f"{name}_traj.csv")
                synth_trajectory_from_labels(local_label, traj, fps=args.fps,
frame_width=args.frame_width)
                specs.append({"name": name, "trajectory": traj, "labels": local_label,
                              "fps": args.fps, "frame_width": args.frame_width})

        if len(specs) < 2:
            print(f"[worker] need >=2 videos with trajectories for leave-one-video-out; got
{len(specs)}.")
            return 2
        manifest_path = os.path.join(scratch, "manifest.json")
        with open(manifest_path, "w", encoding="utf-8") as f:
            json.dump(specs, f, indent=2)
        src_label = "real detector" if args.trajectory_source == "detector" else "CPU-mock
stub"
        print(f"[worker] built manifest: {len(specs)} videos ({src_label} trajectories)")

        out_dir = os.path.join(scratch, "artifacts")
        cmd = [sys.executable, "-m", "training.gen0.harness",
               "--manifest", manifest_path, "--out", out_dir, "--version", args.version]
        if args.floor:
            floor_local = (storage.fetch(args.floor, os.path.join(scratch, "floor.json"),
config=storage_cfg)
                           if "://" in args.floor else args.floor)
            cmd += ["--floor", floor_local]
        print("[worker] training:", " ".join(cmd))
        rc = subprocess.run(cmd).returncode
        if rc != 0:
            print(f"[worker] gen0 harness failed (rc={rc})")
            return rc

        # PUSH the versioned artifact bundle (weights.json + manifest.json) to the bucket.
        bundle = os.path.join(out_dir, args.version)
        out_prefix = args.out_uri.rstrip("/")
        pushed = []
        for fn in sorted(os.listdir(bundle)):
            uri = f"{out_prefix}/weights/{args.version}/{fn}"
            storage.put(os.path.join(bundle, fn), uri, config=storage_cfg)
            pushed.append(uri)
        print(f"[worker] train done: pushed {len(pushed)} artifact(s):")
        for u in pushed:
            print("   ", u)
        return 0


    def build_parser() -> argparse.ArgumentParser:
        p = argparse.ArgumentParser(prog="python -m backend.worker",
                                    description="Storage-aware worker: pull ? run pipeline ?
push.")
        sub = p.add_subparsers(dest="cmd", required=True)

        pi = sub.add_parser("infer", help="run inference on one video URI and push outputs")
```

```
308         pi.add_argument("--video-uri", required=True, help="local path / local:// / s3:// /
gcs://")
309         pi.add_argument("--out-uri", required=True, help="output prefix
(outputs/<video_id>/? is appended)")
310         pi.add_argument("--segmenter", default=None, help="default:
indexing.default_segmenter")
311         pi.add_argument("--config", default=None, help="config.json path (default:
./config.json)")
312         pi.add_argument("--scratch", default=None, help="local scratch dir (default: a temp
dir)")
313         pi.add_argument("--max-frames", type=int, default=None)
314         pi.add_argument("--set", action="append", default=[], metavar="k=v",
315                         help="config override, e.g. --set indexing.skip_ai_handoff=true")
316         pi.set_defaults(func=cmd_infer)
317
318         pt = sub.add_parser("train", help="Gen-0 train-from-bucket (labels [+videos] ->
trajectories -> harness)")
319         pt.add_argument("--corpus-uri", required=True, help="prefix containing labels/ (+
videos/ for "
320                         "--trajectory-source detector), e.g. s3://bucket or gcs://bucket")
321         pt.add_argument("--out-uri", required=True, help="output prefix (weights/<version>/
is appended)")
322         pt.add_argument("--version", required=True, help="semver for the artifact bundle")
323         pt.add_argument("--trajectory-source", choices=["synth", "detector"],
default="synth",
324                         help="synth = CPU-mock label-consistent trajectories (default, no
GPU); "
325                              "detector = REAL shuttle trajectories via the resolved
DetectorRunner "
326                              "(needs videos/ in the corpus; honours detector_impl).")
327         pt.add_argument("--segmenter", default="wasb_hybrid",
328                         help="trajectory hybrid whose detector is used for --trajectory-
source detector")
329         pt.add_argument("--floor", default=None, help="floor baseline JSON (local path or
storage URI)")
330         pt.add_argument("--config", default=None)
331         pt.add_argument("--scratch", default=None)
332         pt.add_argument("--fps", type=float, default=30.0, help="synth trajectory fps
(detector probes the video)")
333         pt.add_argument("--frame-width", type=float, default=1280.0,
334                         help="synth trajectory frame width (detector probes the video)")
335         pt.add_argument("--set", action="append", default=[], metavar="k=v")
336         pt.set_defaults(func=cmd_train)
337         return p
338
339
340     def main(argv=None) -> int:
341         args = build_parser().parse_args(argv)
342         return args.func(args)
343
344
345     if __name__ == "__main__":
346         sys.exit(main())
347
```

19. assistant (2026-06-20T08:58:01.420Z)

I notice `config.get("indexing", {})` is called on line 183 and `_resolve_train_detector` is passed `config` (a Pydantic object from `load_config`). Let me check whether Pydantic config objects support `.get()` ? this is a potential bug but outside byte-parity scope. Let me focus and run the existing cache tests to verify the device param addition doesn't break them, then check config models.

20. user (2026-06-20T08:58:05.625Z)

```
1       """Pydantic models for the application configuration.
2
3       This module defines the canonical, validated, typed configuration for the
4       Badminton Highlight Indexer. Built-in defaults live here (single source of truth).
5
6       The models support graceful migration from old config.json files via extra="allow".
7       """
8
9       from __future__ import annotations
10
11      from typing import Any, List, Literal, Optional
12
13      from pydantic import BaseModel, Field, field_validator, model_validator
14
15      Sport = Literal["badminton", "tennis", "pickleball", "table_tennis", "cricket"]
16
17
18      class ValidationRelevanceConfig(BaseModel):
19          court_green_lower: list[int] = Field(default=[35, 40, 40])
20          court_green_upper: list[int] = Field(default=[85, 255, 255])
21          court_pixels_min_ratio: float = Field(default=0.05, ge=0.0, le=1.0)
22
23
24      class ValidationConfig(BaseModel):
25          min_resolution_width: int = 1280
26          min_resolution_height: int = 720
27          min_fps: float = 29.9
28          min_blur_threshold: float = 20.0
29          max_scene_cuts_per_minute: float = 0.5
30          relevance: ValidationRelevanceConfig =
Field(default_factory=ValidationRelevanceConfig)
31
32
33      class TrackNetConfig(BaseModel):
34          wsl_distro: str = "Ubuntu"
35          repo_dir: str = "~/models/TrackNetV4"
36          conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
37          conda_env: str = "TrackNetV4"
38          weights_path: str = ""
39          queue_length: int = 5
40          stage_in_wsl: bool = True
41          wsl_stage_dir: str = "~/clips"
42          timeout_sec: int = 1800  # 30 min; kills a hung WSL/GPU call (0 = no timeout)
43          velocity_threshold: float = 0.02
44          # Cache hygiene: after a SUCCESSFUL run the staged video copy (and, for WASB, the
45          # decoded frame cache) are deleted automatically ? they are pure intermediates,
46          # regenerable from the source, and the dominant disk consumer (~GBs/video). Set
47          # true only for debugging. On FAILURE they are always kept so a re-run can resume.
48          keep_frames: bool = False
49          # Warn before a run if WSL free space (at wsl_stage_dir) is below this many GB.
50          # 0 = disabled.
51          min_free_gb: float = 0.0
52
53
54      class WasbIndexConfig(BaseModel):
55          """Typed config for the live WASB shuttle substrate (indexing.wasb).
56
57          Mirrors backend/pipeline/detectors/wasb_runner.py::WasbConfig.from_indexing_cfg so
58          these knobs actually take effect ? previously this whole block was silently dropped
59          because nested config sections defaulted to extra='ignore'.
60          """
```

21. user (2026-06-20T08:58:05.856Z)

```
54:class WasbIndexConfig(BaseModel):
```

```
70:    # --- Linux-native runner only (detector_impl='native'); ignored by the WSL runner. ---
72:    device: str = "cuda"        # torch device for the native runner: cuda | mps | cpu
73:    python_bin: str = "python"  # the `wasb` conda env python (invoked by path, no `conda
activate`)
129:    detector_impl: str = "auto"
```

22. user (2026-06-20T08:58:06.095Z)

```
 backend/config/models.py                          |   9 +-
 backend/pipeline/detectors/wasb_infer.py          |  43 ++++++---
 backend/pipeline/segmenters/fusion_hybrid.py      |  13 +++
 backend/pipeline/segmenters/trajectory_hybrid.py  |  28 ++++--
 backend/pipeline/segmenters/wasb_hybrid.py        |   8 ++
 backend/worker/__main__.py                        | 106 ++++++++++++++++++---
 tests/test_worker.py                              |  51 ++++++++++
 7 files changed, 222 insertions(+), 36 deletions(-)
```

23. assistant (2026-06-20T08:58:09.127Z)

Note: `native_wasb_runner.py` and `test_native_wasb_runner.py` aren't in the diff stat ? they're new untracked files. Let me confirm and check the existing wasb_infer cache tests to verify the `device` param doesn't break them.

24. user (2026-06-20T08:58:10.808Z)

```
?? backend/pipeline/detectors/native_wasb_runner.py
?? tests/test_native_wasb_runner.py
---existing wasb_infer tests---
test_calibrate_wasb.py
test_config_wasb.py
test_export_wasb_segments.py
test_native_wasb_runner.py
test_wasb_cache.py
test_wasb_runner.py
```

25. user (2026-06-20T08:58:11.988Z)

```
Found 3 files
tests\test_native_wasb_runner.py
tests\test_wasb_cache.py
tests\test_person_detector.py
```

26. user (2026-06-20T08:58:15.446Z)

```
36:def test_det_plain_and_back_roundtrip():
51:def test_replay_accumulates_detections_across_windows(tmp_path):
68:def test_resume_index_equals_window_count(tmp_path):
76:def test_empty_cache(tmp_path):
86:def test_truncated_trailing_line_recovered_and_cleaned(tmp_path):
105:def test_noncontiguous_window_index_stops_replay(tmp_path):
118:def test_manifest_roundtrip_and_compat(tmp_path):
137:    assert wi.manifest_compatible(m, **base)
138:    assert not wi.manifest_compatible(m, **{**base, "frames_total": 101})   # more frames
139:    assert not wi.manifest_compatible(m, **{**base, "weights": "other.pth"})
140:    assert not wi.manifest_compatible(m, **{**base, "sport": "tennis"})
141:    assert not wi.manifest_compatible({}, **base)                           # no cache
142:    assert not wi.manifest_compatible({**m, "version": 999}, **base)       # schema bump
145:def test_read_manifest_missing_returns_none(tmp_path):
149:def test_reset_cache_removes_files(tmp_path):
162:def test_default_cache_dir_is_next_to_output():
169:def test_atomic_write_trajectory(tmp_path):
183:def test_synthetic_frame_path_roundtrip():
193:def test_build_windows_matches_disk_logic():
203:def test_build_windows_too_few_frames_is_empty():
```

```
221:def test_store_reads_each_frame_once_in_order_for_sliding_windows():
234:def test_store_evicts_outside_window_bounded_memory():
246:def test_store_keeps_window_minus_one_lookback():
257:def test_store_resume_starts_at_start_index_without_decoding_prefix():
268:def test_store_out_of_range_raises():
286:def test_make_streamed_read_image_returns_pil_rgb_and_falls_through():
310:def test_make_streamed_read_image_resume_primes_start_index():
324:def test_bgr_to_pil_rgb_matches_disk_roundtrip(tmp_path):
```

27. assistant (2026-06-20T08:58:18.128Z)

Let me look at the critical manifest compatibility test (lines 118-142) to verify the device-keying change is backward compatible with these tests.

28. user (2026-06-20T08:58:19.212Z)

```
118     def test_manifest_roundtrip_and_compat(tmp_path):
119         cache = str(tmp_path / "c")
120         os.makedirs(cache)
121         m = {
122             "version": wi.CACHE_VERSION,
123             "frames_dir": osp.abspath("/x/frames"),
124             "weights": "w.pth.tar",
125             "sport": "badminton",
126             "frames_in": 3,
127             "frames_total": 100,
128             "windows_total": 98,
129             "windows_done": 42,
130         }
131         wi.write_manifest(cache, m)
132         assert not osp.exists(osp.join(cache, wi._MANIFEST_FILE + ".tmp"))   # atomic, no
tmp left
133         assert wi.read_manifest(cache) == m
134 
135         base = dict(frames_dir="/x/frames", weights="w.pth.tar", sport="badminton",
136                     frames_in=3, frames_total=100)
137         assert wi.manifest_compatible(m, **base)
138         assert not wi.manifest_compatible(m, **{**base, "frames_total": 101})   # more
frames
139         assert not wi.manifest_compatible(m, **{**base, "weights": "other.pth"})
140         assert not wi.manifest_compatible(m, **{**base, "sport": "tennis"})
141         assert not wi.manifest_compatible({}, **base)                           # no cache
142         assert not wi.manifest_compatible({**m, "version": 999}, **base)       # schema bump
143 
144 
145     def test_read_manifest_missing_returns_none(tmp_path):
146         assert wi.read_manifest(str(tmp_path)) is None
147 
148 
149     def test_reset_cache_removes_files(tmp_path):
150         cache = str(tmp_path / "c")
151         os.makedirs(cache)
152         _write_windows(cache, [_win(0, [(0, [[1, 1, 0.9, 0]])])])
153         wi.write_manifest(cache, {"version": wi.CACHE_VERSION})
154         wi.reset_cache(cache)
155         det_jsonl, mpath = wi._cache_paths(cache)
156         assert not osp.exists(det_jsonl) and not osp.exists(mpath)
157 
158 
159     # ---------------------------------------------------------------------- #
160     # misc helpers
161     # ---------------------------------------------------------------------- #
162     def test_default_cache_dir_is_next_to_output():
```

29. assistant (2026-06-20T08:58:24.618Z)

This confirms the back-compat is correct: `test_manifest_roundtrip_and_compat` builds a manifest `m` with **no `device` field** and calls `manifest_compatible(m, **base)` with **no `device` kwarg** (line 137). Since both default to `"cuda"`, the comparison `manifest.get("device", "cuda") == device` ? `"cuda" == "cuda"` ? `True`. The existing test still passes. The device param default preserves backward compatibility cleanly.

Now let me verify the most important divergence ? let me carefully re-examine the WSL vs native `--out` path handling and confirm both produce the identical CSV *filename and content* on a cuda run. Let me run the new tests to confirm they actually pass and check the worker test.

30. user (2026-06-20T08:58:33.052Z)

```
............................................................           [100%]
62 passed in 5.50s
```

31. assistant (2026-06-20T08:58:43.579Z)

All 62 tests pass. Now let me carefully examine the **one remaining subtle byte-parity risk**: the WSL runner extracts frames from the **staged copy** of the video, while the native runner extracts from the **original** video at its `--video` path. Both use `extract_frames` (cv2) ? identical PNGs since it's the same video bytes. But there's a subtler concern: `extract_frames` is **idempotent** based on frame count ? if the native `frames_dir` (in scratch) already has a stale partial extraction from a different video sharing the key, it could reuse wrong frames. But the key is `{stem}__{vid12}` (content-derived), so that's safe. Good.

Let me now check one thing I flagged: the `--video` path. WSL passes the **staged** path (a copy in `~/clips`). Native passes `os.path.abspath(str(video_win_path))`. On Windows tests, `os.path.abspath` of `C:\v\clip.mp4` works, but on Linux (where it actually runs) the video path is a native Linux path. Let me verify the abspath handling and check that `--video` is correctly consumed. Let me also examine the streaming path's `source_key` ? this is where a real parity divergence could hide.

For streaming: `source_key = "video:" + osp.abspath(video_path)`. WSL streams the **staged** video (`~/clips/{key}{ext}`) ? `source_key = video:/home/user/clips/...`. Native streams the **original** path ? `source_key = video:/native/path/...`. These differ ? the streaming manifest cache key differs WSL?native, but again only affects resume, not fresh CSV bytes. The trajectory is identical because the same video bytes are decoded. Good.

Now let me check the trajectory_hybrid `_resolve_runner` and fusion_hybrid diffs more carefully, and look at one concern: `config.get("indexing", {})` in the worker on a Pydantic object.

32. user (2026-06-20T08:58:45.481Z)

```
1       """Multi-signal fusion segmenter (MULTI_SIGNAL_FUSION_PLAN P2).
2
3       Shuttle stays PRIMARY: this reuses an existing trajectory substrate (WASB by default,
4       TrackNet optional) to seed candidate windows exactly as `wasb_hybrid`/`tracknet_hybrid`
5       do ? it subclasses the same `TrajectoryHybridSegmenter` engine. It then ENRICHES each
6       shuttle-seeded window via two overridable seams the engine exposes:
7
8       - ``_enrich_candidate_stats`` ? always adds the **net-crossing count** (Gate A signal,
9         GPU-free, computed from the trajectory we already have); and, only when the person cue
10        is explicitly enabled (``indexing.fusion.cue_flags.person``), the person-cue features
11        (`fusion_features`), persisting them into ``candidate_segments.stats`` for the learned
12        fusion (P3). The person path lazily builds ONE `PersonDetector` and only then loads
the
13        torchvision/supervision model ? so the default (person-OFF) path never loads a
detector
14        model nor touches the GPU. (torch/cv2 themselves are imported by the registry
regardless,
15        via the pre-existing yolo_hybrid ? person_detector chain; the lazy part is the model.)
16      - ``_select_for_handoff`` ? optional served Gate A/B: when
```

```
``indexing.fusion.min_crossings``
17        (and, with the person cue, ``min_players``) are raised above 0, sub-threshold windows
are
18        dropped from the AI handoff (the held-shuttle / one-sided-feed false-positive fix).
19        **Persisted candidates remain the full superset** regardless, so the offline
experiment
20        harness can re-score any variant.
21
22      Defaults reproduce the substrate exactly: net_crossings/person features are persisted
but
23      nothing is gated (thresholds 0, person OFF), so `FusionSegmenter` with default config
seeds
24      and hands off identically to `wasb_hybrid`. Registered as ``fusion_hybrid``, NOT the
default
25      segmenter ? it is opt-in. Promotion to default happens only on measured LOVO lift
through the
26      experiment harness (EXPERIMENT_HARNESS.md), never silently.
27      """
28      from typing import Any, Dict, List, Optional, Tuple
29
30      from backend.pipeline.segmenters.base import segmenter_registry
31      from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
32      from backend.pipeline.detectors.base import DetectorRunner
33      from backend.pipeline.detectors.wasb_runner import WasbRunner, WasbConfig
34      from backend.pipeline.detectors.tracknet_runner import TrackNetRunner, TrackNetConfig
35      from backend.pipeline.detectors import rally_gate
36      from backend.pipeline.detectors import fusion_features as ff
37
38
39      class FusionSegmenter(TrajectoryHybridSegmenter):
40          """Shuttle trajectory (primary) + net-crossing Gate A + optional person cue (Gate
B)."""
41
42          family = "fusion"
43          display_label = "Fusion"
44
45          def __init__(self, db: Any, config: Any):
46              super().__init__(db, config)
47              # Built lazily only if the person cue is enabled (no detector model loaded
otherwise).
48              self._person: Optional[Any] = None
49              # Cache the play-axis/net estimate per trajectory so rally_gate doesn't re-
estimate it
50              # over the whole trajectory once per candidate window (it depends only on
`points`).
51              self._axis_cache_key: Optional[int] = None
52              self._axis_cache: Optional[tuple] = None
53
54          @property
55          def name(self) -> str:
56              return "fusion_hybrid"
57
58          @property
59          def description(self) -> str:
60              return ("Multi-signal fusion: shuttle trajectory (WASB/TrackNet) seeds candidate
rallies, "
61                      "net-crossing Gate A + optional person-occupancy Gate B adjudicate them,
and the "
62                      "AI provider sets semantic boundaries.")
63
64          def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
Any]]:
65              substrate = idx_cfg.get("fusion", {}).get("substrate", "wasb")
66              if substrate == "tracknet":
67                  cfg = TrackNetConfig.from_indexing_cfg(idx_cfg)
```

```
68                  return TrackNetRunner(cfg), {
69                      "weights_path": cfg.weights_path,
70                      "queue_length": cfg.queue_length,
71                      "fusion_substrate": "tracknet",
72                  }
73              wcfg = WasbConfig.from_indexing_cfg(idx_cfg)
74              return WasbRunner(wcfg), {"weights_path": wcfg.weights_path, "fusion_substrate":
"wasb"}
75
76          def _build_native_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner,
Dict[str, Any]]:
77              # Native (no-WSL) path supports the WASB substrate only ? there is no native
TrackNet
78              # runner yet. Fusion's default substrate is WASB, so this covers the common
case.
79              substrate = idx_cfg.get("fusion", {}).get("substrate", "wasb")
80              if substrate != "wasb":
81                  raise NotImplementedError(
82                      "detector_impl='native' supports the WASB substrate only ? set "
83                      "indexing.fusion.substrate='wasb' (no native TrackNet runner yet).")
84              from backend.pipeline.detectors.native_wasb_runner import NativeWasbRunner,
NativeWasbConfig
85              cfg = NativeWasbConfig.from_indexing_cfg(idx_cfg)
86              return NativeWasbRunner(cfg), {"weights_path": cfg.weights_path,
"detector_impl": "native",
87                                             "fusion_substrate": "wasb", "device": cfg.device}
88
89          def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
90                                      frame_width: float, video_path: str,
```

33. user (2026-06-20T08:58:46.496Z)

```
diff --git a/backend/pipeline/segmenters/fusion_hybrid.py
b/backend/pipeline/segmenters/fusion_hybrid.py
index 953ea4b..8518674 100644
--- a/backend/pipeline/segmenters/fusion_hybrid.py
+++ b/backend/pipeline/segmenters/fusion_hybrid.py
@@ -73,6 +73,19 @@ class FusionSegmenter(TrajectoryHybridSegmenter):
         wcfg = WasbConfig.from_indexing_cfg(idx_cfg)
         return WasbRunner(wcfg), {"weights_path": wcfg.weights_path, "fusion_substrate":
"wasb"}

+    def _build_native_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
Any]]:
+        # Native (no-WSL) path supports the WASB substrate only ? there is no native TrackNet
+        # runner yet. Fusion's default substrate is WASB, so this covers the common case.
+        substrate = idx_cfg.get("fusion", {}).get("substrate", "wasb")
+        if substrate != "wasb":
+            raise NotImplementedError(
+                "detector_impl='native' supports the WASB substrate only ? set "
+                "indexing.fusion.substrate='wasb' (no native TrackNet runner yet).")
+        from backend.pipeline.detectors.native_wasb_runner import NativeWasbRunner,
NativeWasbConfig
+        cfg = NativeWasbConfig.from_indexing_cfg(idx_cfg)
+        return NativeWasbRunner(cfg), {"weights_path": cfg.weights_path, "detector_impl":
"native",
+                                       "fusion_substrate": "wasb", "device": cfg.device}
+
     def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
                                 frame_width: float, video_path: str,
                                 idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
diff --git a/backend/pipeline/segmenters/trajectory_hybrid.py
b/backend/pipeline/segmenters/trajectory_hybrid.py
index 9b1f202..f7edb9d 100644
--- a/backend/pipeline/segmenters/trajectory_hybrid.py
```

```
+++ b/backend/pipeline/segmenters/trajectory_hybrid.py
@@ -51,17 +51,27 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
     display_label: str = ""

     def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str, Any]]:
-        """Return (runner, detector-specific detection_params extras)."""
+        """Return (runner, detector-specific detection_params extras) for the default (WSL)
path."""
         raise NotImplementedError

+    def _build_native_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
Any]]:
+        """Return (runner, extras) for the Linux-native (no-WSL) path. Only families with a
+        native runner override this; the base refuses with a clear message (M3.5: WASB
only)."""
+        raise NotImplementedError(
+            f"detector_impl='native' is not supported for the "
+            f"{self.display_label or self.family or 'trajectory'!r} family ? only WASB has a "
+            f"Linux-native runner. Use detector_impl='auto' (WSL) or 'stub'.")
+
     def _resolve_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
Any]]:
-        """Resolve the detector runner, honouring the CPU-stub override.
+        """Resolve the detector runner, honouring the CPU-stub and Linux-native overrides.

         ``RALLY_DETECTOR_IMPL`` env var (takes precedence) or ``indexing.detector_impl``:
         ``"stub"`` swaps in a deterministic CPU runner (no GPU/WSL) so the whole pipeline
-        is regression-testable without a GPU ? "mock a GPU with a CPU"
-        (``docs/DECOUPLED_COMPUTE/``). Anything else delegates to ``_build_runner`` (the
-        real WSL/native runner) so the default production path is unchanged.
+        is regression-testable without a GPU ? "mock a GPU with a CPU"; ``"native"`` swaps
+        in the Linux-native WASB runner (no WSL/conda-activate; GPU box) via
+        ``_build_native_runner``. Anything else (``"auto"``) delegates to ``_build_runner``
+        (the WSL runner) so the default production path is unchanged
+        (``docs/DECOUPLED_COMPUTE/``).
         """
         impl = (os.environ.get("RALLY_DETECTOR_IMPL") or idx_cfg.get("detector_impl") or
"auto").lower()
         if impl == "stub":
@@ -69,6 +79,10 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
             logger.info("%s: detector_impl=stub ? CPU mock runner (no GPU/WSL).",
                         self.display_label or "trajectory")
             return StubDetectorRunner(StubConfig.from_indexing_cfg(idx_cfg)), {"detector_impl":
"stub"}
+        if impl in ("native", "native_wasb", "wasb_native"):
+            logger.info("%s: detector_impl=native ? Linux-native runner (no WSL).",
+                        self.display_label or "trajectory")
+            return self._build_native_runner(idx_cfg)
         return self._build_runner(idx_cfg)

     @staticmethod
@@ -206,9 +220,9 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
         # --- Phase 1: env healthcheck (fail fast with a clear message) ---
         ok, msg = runner.healthcheck()
         if not ok:
-            logger.error("%s WSL environment not ready: %s", label, msg)
+            logger.error("%s detector environment not ready: %s", label, msg)
             return []
-        logger.info("%s WSL env OK: %s", label, msg)
+        logger.info("%s detector env OK: %s", label, msg)

         # --- Phase 2: trajectory -> candidate rally windows ---
         # Trajectory detectors process the whole video in one pass (they carry
```

34. assistant (2026-06-20T08:58:52.937Z)

The hybrid wiring is clean. Now let me dig into the one byte-parity divergence I'm most concerned about: the **`detector.model_path` / `--weights` value baked into the Hydra override and the manifest**. Let me also re-examine whether `extract_frames` writing to `frames_out_dir` is consumed correctly when `--video` + `--frames_out_dir` are both passed (native always passes `--video`, WSL also passes `--video`).

Let me check the WASB Hydra config to confirm whether `runner.device` actually controls tensor placement, and whether there's any device-dependent nondeterminism (e.g., cudnn) ? but that's a hardware concern the owner verifies. Let me focus on the code-level argv comparison precisely.

Let me build a precise side-by-side of the argv for the **default disk + cuda** path, which is the parity gate.

35. user (2026-06-20T08:58:59.003Z)

```
=== WSL runner (wasb_runner.py), default disk + cuda ===
weights_path default: ~/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar
cmd (one bash string):
  conda activate wasb && cd ~/models/WASB-SBDT/src && python wasb_infer.py \
    --video '~/clips/{key}{ext}' \
    --frames_out_dir '~/clips/{key}_frames' \
    --weights '~/models/.../wasb_badminton_best.pth.tar' \
    --sport badminton \
    --out '/mnt/c/.../output/wasb/{key}_wasb.csv'
  (NOTE: NO --device flag passed -> wasb_infer default device=cuda)

=== Native runner (native_wasb_runner.py), default disk + cuda ===
weights_path: expanduser(cfg.weights_path)  (REQUIRED, no default file)
argv (list):
  [python, wasb_infer.py,
   --video <abspath(orig video)>,
   --frames_out_dir <scratch-or-outdir>/{key}_frames,
   --weights <expanded weights>,
   --sport badminton,
   --device cuda,
   --out <outdir>/{key}_wasb.csv]
  cwd = expanduser(repo_dir)/src
```

36. assistant (2026-06-20T08:59:18.391Z)

This is the crux. Let me now reason through each byte-affecting input to `wasb_infer.py run()`:

1. **`weights`** ? goes into Hydra `detector.model_path={weights}` AND `manifest["weights"]`. WSL passes the **literal `~/...` string** (Hydra/torch load resolves `~` at load ? actually, does torch `load_state_dict`/`torch.load` expand `~`? Typically NO ? `torch.load("~/...")` would fail). Let me check this carefully. If WSL passes a literal `~/` weights string and it actually loads, then Hydra/WASB must be expanding it, OR the bash `~` is NOT expanded because `wsl_tilde_quote` produces `~/'models/...'`. Bash WOULD expand the leading `~/`. So WSL's `argv` to python receives the **already-expanded absolute** weights path (bash expanded it before exec). Native passes `os.path.expanduser` ? also absolute. **So both pass an absolute weights path to wasb_infer ? identical `detector.model_path` ? identical model.** Parity holds. Good ? this is actually fine.

2. **frames** ? identical PNGs from identical video bytes ? identical sorted list ? identical tensors. Parity holds.

3. **`device`** ? WSL passes no `--device` (default cuda); native passes `--device cuda`. Both ? `device="cuda"` ? identical Hydra `runner.device=cuda`, `runner.gpus=[0]`. Parity holds. ? (This is the key contract and it's correct.)

4. **`sport`, `out` filename** ? identical. ?

5. **cache/manifest** ? `frames_dir`/`source_key` differ (different absolute paths), and `weights` field differs only if the resolved abspaths differ. These affect **resume only**, not fresh CSV. ?

So the **byte-parity contract is upheld** for the core. The divergences I found are all in resume/cache identity (intended, doesn't affect a fresh CSV) ? but there IS one real concern worth flagging precisely. Let me verify the weights-expansion reasoning by checking how WASB actually loads the model path, and check whether the manifest `weights` difference could cause a **cross-contamination** bug (cpu reusing cuda cache).

Let me re-read `manifest_compatible` for the cross-device cache safety claim and verify the cpu-vs-cuda scenario precisely.

The manifest line: `manifest.get("device", "cuda") == device`.

- **Old cuda cache (no device field) + cuda run**: `"cuda" == "cuda"` ? resume. ? (back-compat preserved)
- **Old cuda cache (no device field) + cpu run**: `"cuda" == "cpu"` ? False ? no resume. ? (cpu won't reuse cuda cache)
- **New cpu cache (device="cpu") + cuda run**: `"cpu" == "cuda"` ? False ? no resume. ?

This is correct. But there's a subtle edge: **a cuda cache and a native-cuda cache share the same device value AND, if `frames_dir`/`weights` happen to match, would cross-resume.** Since the native frames_dir is a different abspath than WSL's `~/clips`, they won't collide in practice. But within native itself (cuda), a resume is fine. No bug.

Now let me check one final real concern: does the native runner's `--video` + `--frames_out_dir` correctly trigger extraction, and is there any issue with `extract_frames` default when `--frames_out_dir` is empty string? Native always sets it (non-stream). Let me verify the worker test and run the broader test suite to ensure nothing regressed.

37. user (2026-06-20T08:59:19.528Z)

```
1       """Worker dispatcher: pull (storage) ? run pipeline ? push (storage).
2
3       `cmd_infer` is exercised end-to-end over the LOCAL storage backend with the validator +
4       segmenter mocked, so it needs no real video/GPU/network. The real S3 round-trip is
validated
5       on the droplet against MinIO.
6       """
7       import json
8
9       import pytest
10
11      from backend.config import load_config
12      from backend.worker import __main__ as wmain
13
14
15      # ---------------------------------------------------------------- pure helpers
16
17      @pytest.mark.parametrize("raw,expected", [
18          ("true", True), ("False", False), ("3", 3), ("2.5", 2.5), ("wasb_hybrid",
"wasb_hybrid"),
19      ])
20      def test_coerce(raw, expected):
21          assert wmain._coerce(raw) == expected and type(wmain._coerce(raw)) is type(expected)
22
23
24      def test_apply_overrides_on_typed_config():
25          cfg = _fresh_config()
26          wmain._apply_overrides(cfg, ["indexing.skip_ai_handoff=true",
"indexing.min_segment_duration=1.5",
27                                       "sport=tennis"])
28          assert cfg.indexing.skip_ai_handoff is True
29          assert cfg.indexing.min_segment_duration == 1.5
30          assert cfg.sport == "tennis"
31
32
33      def test_apply_overrides_rejects_bad_format():
```

```
34          cfg = _fresh_config()
35          with pytest.raises(ValueError):
36              wmain._apply_overrides(cfg, ["no_equals_sign"])
37
38
39      def test_write_intervals_csv(tmp_path):
40          segs = [{"start_time": 2.0, "end_time": 5.0, "shot_count": 4,
"shot_count_confidence": "LocalNoAI"},
```

38. assistant (2026-06-20T08:59:23.063Z)

Let me check the worker test for the `_resolve_train_detector` path and verify the `config.get("indexing", {})` concern ? whether `config` is a Pydantic object (no `.get()`) or a dict in the train path.

39. user (2026-06-20T08:59:24.428Z)

```
147:# --- cmd_train: pull labels -> synth trajectories -> manifest -> harness -> push bundle ---
149:def test_cmd_train_orchestration(tmp_path, monkeypatch):
191:    assert args.trajectory_source == "detector" and args.segmenter == "fusion_hybrid"
194:def test_cmd_train_detector_source_runs_real_extract_via_stub(tmp_path, monkeypatch):
```

40. user (2026-06-20T08:59:27.057Z)

```
147     # --- cmd_train: pull labels -> synth trajectories -> manifest -> harness -> push bundle
---
148
149     def test_cmd_train_orchestration(tmp_path, monkeypatch):
150         import os
151         import subprocess
152
153         labels_dir = tmp_path / "corpus" / "labels"
154         labels_dir.mkdir(parents=True)
155         header = "rally_number,start_time,end_time,ending_reason,sport,shots_count\n"
156         (labels_dir / "v1.rallies.csv").write_text(header +
"1,2.0,5.0,winner,badminton,\n2,9.0,12.0,let,badminton,\n")
157         (labels_dir / "v2.rallies.csv").write_text(header + "1,3.0,6.0,winner,badminton,\n")
158         out_dir = tmp_path / "out"
159         scratch = tmp_path / "scratch"
160         cfg = tmp_path / "config.json"
161         cfg.write_text("{}")
162
163         # Mock the harness subprocess: write a fake versioned bundle at --out/--version.
164         def fake_run(cmd, *a, **k):
165             out = cmd[cmd.index("--out") + 1]
166             ver = cmd[cmd.index("--version") + 1]
167             bdir = os.path.join(out, ver)
168             os.makedirs(bdir, exist_ok=True)
169             for fn in ("weights.json", "manifest.json"):
170                 with open(os.path.join(bdir, fn), "w") as f:
171                     f.write("{}")
172             return subprocess.CompletedProcess(cmd, 0)
173         monkeypatch.setattr(subprocess, "run", fake_run)
174
175         rc = wmain.main(["train", "--corpus-uri", str(tmp_path / "corpus"), "--out-uri",
str(out_dir),
176                          "--version", "0.0.1", "--config", str(cfg), "--scratch",
str(scratch)])
177         assert rc == 0
178         # bundle pushed to <out-uri>/weights/<version>/
179         assert (out_dir / "weights" / "0.0.1" / "weights.json").exists()
180         assert (out_dir / "weights" / "0.0.1" / "manifest.json").exists()
181         # the manifest fed to the harness enumerated both labeled videos with a trajectory
each
```

```
182         specs = json.loads((scratch / "manifest.json").read_text())
183         assert len(specs) == 2
184         assert all("trajectory" in s and "labels" in s and "fps" in s for s in specs)
185
186
187     def test_parser_train_detector_args():
188         args = wmain.build_parser().parse_args([
189             "train", "--corpus-uri", "gcs://b", "--out-uri", "gcs://b", "--version",
"1.0.0",
190             "--trajectory-source", "detector", "--segmenter", "fusion_hybrid"])
191         assert args.trajectory_source == "detector" and args.segmenter == "fusion_hybrid"
192
193
194     def test_cmd_train_detector_source_runs_real_extract_via_stub(tmp_path, monkeypatch):
195         """--trajectory-source detector pulls videos + runs the resolved DetectorRunner.
With
196         RALLY_DETECTOR_IMPL=stub the *wiring* is exercised GPU-free (the stub IS the
runner)."""
197         import os
198         import subprocess
199
200         monkeypatch.setenv("RALLY_DETECTOR_IMPL", "stub")  # resolve a CPU stub runner (no
GPU/WSL)
201         corpus = tmp_path / "corpus"
202         labels_dir = corpus / "labels"
203         videos_dir = corpus / "videos"
204         labels_dir.mkdir(parents=True)
205         videos_dir.mkdir(parents=True)
206         header = "rally_number,start_time,end_time,ending_reason,sport,shots_count\n"
207         (labels_dir / "v1.rallies.csv").write_text(header + "1,2.0,5.0,winner,badminton,\n")
208         (labels_dir / "v2.rallies.csv").write_text(header + "1,3.0,6.0,winner,badminton,\n")
209         (videos_dir / "v1.mp4").write_bytes(b"FAKEVIDEO1")
210         (videos_dir / "v2.mp4").write_bytes(b"FAKEVIDEO2")
211         out_dir = tmp_path / "out"
212         scratch = tmp_path / "scratch"
213         cfg = tmp_path / "config.json"
214         cfg.write_text("{}")
215
216         def fake_run(cmd, *a, **k):
217             out = cmd[cmd.index("--out") + 1]
218             ver = cmd[cmd.index("--version") + 1]
219             bdir = os.path.join(out, ver)
220             os.makedirs(bdir, exist_ok=True)
221             for fn in ("weights.json", "manifest.json"):
222                 with open(os.path.join(bdir, fn), "w") as f:
223                     f.write("{}")
224             return subprocess.CompletedProcess(cmd, 0)
225         monkeypatch.setattr(subprocess, "run", fake_run)
226
227         rc = wmain.main(["train", "--corpus-uri", str(corpus), "--out-uri", str(out_dir),
228                          "--version", "0.0.2", "--config", str(cfg), "--scratch",
str(scratch),
229                          "--trajectory-source", "detector", "--segmenter", "wasb_hybrid"])
230         assert rc == 0
231         specs = json.loads((scratch / "manifest.json").read_text())
232         assert len(specs) == 2
233         # Real-extract path: each spec points at an on-disk trajectory the runner actually
wrote.
234         assert all(os.path.exists(s["trajectory"]) for s in specs)
235         assert all(s["trajectory"].endswith("_stub.csv") for s in specs)  # the stub
runner's output
236
```

41. assistant (2026-06-20T08:59:32.325Z)

Now there's a real concern in `_resolve_train_detector`: line 183 calls `config.get("indexing", {})` but `config` is a Pydantic model (from `load_config`), which has no `.get()`. The test passes only because `RALLY_DETECTOR_IMPL=stub` is set as env var ? and `_resolve_runner` reads the env var FIRST, then `idx_cfg.get("detector_impl")`. Let me trace: `_resolve_runner(config.get("indexing", {}))` ? `config.get(...)` would raise `AttributeError` on a Pydantic object BEFORE `_resolve_runner` runs. But the test passed (rc==0)... Let me verify whether the Pydantic config supports `.get()`.

42. user (2026-06-20T08:59:34.612Z)

```
type: <class 'backend.config.models.AppConfig'>
has .get: True
c.get(indexing): <class 'dict'>
```

43. assistant (2026-06-20T08:59:37.513Z)

Interesting ? the Pydantic config has a `.get()` method (likely added to AppConfig), and `c.get("indexing", {})` returns a dict. So `_resolve_train_detector` works. Let me confirm that `.get("indexing")` returns a plain dict so `NativeWasbConfig.from_indexing_cfg` (which calls `.get("wasb", {})`) works on it.

44. user (2026-06-20T08:59:40.537Z)

```
indexing is dict: True
wasb sub: <class 'dict'> {'wsl_distro': 'Ubuntu', 'repo_dir': '~/models/WASB-SBDT', 'conda_sh':
'~/miniconda3/etc/profile.d/conda.sh', 'conda_env': 'wasb', 'weights_path': '~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar', 'sport': 'badminton', 'wsl_stage_dir':
'~/clips', 'timeout_sec': 1800, 'velocity_threshold': 0.02, 'device': 'cuda', 'python_bin':
'python', 'keep_frames': False, 'min_free_gb': 0.0, 'stream_video': False}
native cfg device/python/weights: cuda python '~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar'
```

45. assistant (2026-06-20T08:59:50.940Z)

Now I found a **genuine byte-parity-relevant divergence**. The config default `weights_path` is the tilde-relative path `~/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar`.

- **WSL runner**: passes weights via `wsl_tilde_quote` ? bash expands `~/` ? wasb_infer gets the **absolute** path ? `detector.model_path=/home/user/models/...`.
- **Native runner**: `os.path.expanduser(self.cfg.weights_path)` ? on Linux GPU box, `$HOME` for the native user ? also absolute, and if `$HOME` is the same Linux user, it's the **same absolute path**. So `detector.model_path` matches ? model identical ? parity holds.

This is fine **as long as the same file is at that path**. So the weights path is NOT a divergence in the trajectory bytes. Good.

But here's the **real subtle issue I want to verify**: the native `from_indexing_cfg` reads `device=env("WASB_DEVICE") or w.get("device", "cuda")`. The native config's default `weights_path` is `""` (empty, REQUIRED) per the dataclass, but when built from the real config's `indexing.wasb` block, it picks up the WSL default `~/models/.../wasb_badminton_best.pth.tar`. That's actually convenient (shares the WSL config). No divergence.

Let me now run the full relevant test groups (config, worker, segmenter resolution) plus ruff on the changed files to confirm no regressions, then finalize.

46. user (2026-06-20T09:00:00.873Z)

```
.............................................                            [100%]
48 passed in 6.75s
```

47. user (2026-06-20T09:00:01.113Z)

```
All checks passed!
```

48. assistant (2026-06-20T09:00:09.382Z)

All green. Now let me verify the one remaining edge that could be a real divergence: **the `--frames_out_dir` location for native when `scratch_dir` is empty defaults to `output_dir`**, while WSL uses `~/clips`. This means native extracts PNGs *into the output dir* (next to the CSV). This is purely a frame-cache location difference ? identical PNGs, identical glob/sort ? identical CSV. Not a divergence in bytes. But let me confirm `extract_frames` glob behavior doesn't pick up unrelated files in the output dir. `extract_frames` writes to `{key}_frames/` subdir, and `run()` globs `{frames_dir}/*.png` where `frames_dir` is that subdir. The CSV `{key}_wasb.csv` lives in the parent `output_dir`, NOT inside `{key}_frames/`. So no stray-file contamination. Safe.

Let me do a final check on one thing the prompt explicitly asked: does adding `--device`/`device` params break any **existing pure-Python cache tests**? I confirmed `test_manifest_roundtrip_and_compat` passes. Let me run the entire `test_wasb_cache.py` once more explicitly and grep for any direct call sites of `run()`/`run_video_streaming()` in tests that pass positional args that could now be misaligned by the new `device` kwarg.

49. user (2026-06-20T09:00:11.194Z)

tests\test_wasb_cache.py:19:from backend.pipeline.detectors import wasb_infer as wi

50. assistant (2026-06-20T09:00:20.466Z)

No test calls `run()`/`run_video_streaming()` with positional args that the new `device` kwarg (appended last, after `frame_loader`/`source_key`) could displace. The `device` param is added at the **end** of every signature with a default, so positional callers are unaffected. The cache tests only exercise the pure-Python helpers (`manifest_compatible`, `load_and_clean_cache`, etc.), and `manifest_compatible`'s new `device` kwarg defaults to `"cuda"` matching the legacy no-field manifest. Confirmed safe.

I have thoroughly traced every byte-affecting input. Let me compile my findings.

Summary of analysis: The byte-parity contract on `device=cuda` is **upheld**. The `wasb_infer.py` changes preserve the exact Hydra overrides for cuda (`runner.device=cuda`, `runner.gpus=[0]`), the native argv runs the same script with the same `--out` filename, same cwd (repo `src/`), same weights/frames/sport. All divergences I found are confined to the **resume cache identity** (intended) and do not affect a fresh-run CSV. The device-keyed manifest is correctly backward-compatible. No existing pure-Python cache tests break.

Let me note the items worth flagging (none are blockers to byte-parity, but a couple are worth the owner's attention as nits/minor for the GPU-box parity run).

51. user (2026-06-20T09:00:45.541Z)

Structured output provided successfully

52. assistant (2026-06-20T09:00:59.710Z)

I completed the byte-parity skeptic review. My verdict: **the byte-parity contract on `device=cuda` holds** for the fresh-run trajectory CSV. I traced every byte-affecting input to `wasb_infer.py::run()` and found no divergence that changes the CSV or the candidate windows.

Key confirmations:
- **argv parity**: native runs the same `python wasb_infer.py` with the same `--out {key}_wasb.csv` filename, same cwd (repo `src/`, so Hydra's relative `config_path="configs"` resolves identically), same `--weights`/`--sport`/frames. The only argv difference is native explicitly passing `--device cuda` vs WSL relying on the default ? both resolve to `device="cuda"`.
- **Hydra override parity**: `_build_cfg`'s `device="cuda"` default produces exactly the pre-change overrides (`runner.device=cuda`, `runner.gpus=[0]`). The `gpus = "[0]" if device=="cuda" else "[]"` branch only changes behavior for cpu/mps. `git diff` confirms cuda is untouched.
- **weights**: WSL's `wsl_tilde_quote` leaves `~/` for bash to expand, so wasb_infer receives an absolute weights path; native uses `os.path.expanduser` ? both pass the same absolute path ?

same `detector.model_path` ? same model.
- **frames**: identical video bytes ? identical cv2 PNGs ? identical `sorted(glob)` ? identical tensors.

The divergences I did find are all confined to the **resumable-cache identity** (manifest `frames_dir`/`source_key`/`weights`), which only governs whether a *prior* cache resumes ? never the bytes of a fresh CSV. A WSL cache and a native cache won't cross-resume (different abspaths), so each recomputes ? identical output.

On the explicit questions asked:
- **Old cuda cache (no `device` field) on a cuda run**: resumes correctly (`manifest.get("device","cuda")=="cuda"`). ?
- **cpu run reusing a cuda cache**: blocked (`"cuda" != "cpu"`). ?
- **Existing pure-Python cache tests**: not broken ? `device` is appended last with a default; `test_manifest_roundtrip_and_compat` (device-less manifest, no device kwarg) still passes because both default to `"cuda"`. Verified by running 62 detector/cache/worker tests green, plus ruff clean on all changed files.

Findings returned via StructuredOutput: one explanatory nit (cache-key differences are expected/benign), one minor (the device field is the sole cross-device guard ? fine today, don't rely on it as stronger than it is), and one nit (native disk-mode extracts frames next to the CSV with no `min_free_gb` equivalent). None are byte-parity blockers.